

Lustjinn

Python as a backend, for a .NET developer

A course that grows with the code. This edition was built on 14 October 2026.

Contents

0 Orientation 5
- 0.1 How to read this course 5
- 0.2 The map 5
- 0.3 Break 1 — Types are checked, but not enforced 6
- 0.4 Break 2 — There is no build step 7
- 0.5 Break 3 — Every project gets its own interpreter environment 7
- 0.6 Break 4 — Async runs on one thread 8
- 0.7 Smaller differences you will meet soon 8
- 0.8 What is already on your machine 8
- 0.9 What is next 9

1 Hello, deployed 10
- 1.1 What uv created 10
- 1.2 The project file 11
- 1.3 The app 12
- 1.4 How Python is served 12
- 1.5 The test 13
- 1.6 The gates 14
- 1.7 CI on GitHub Actions 14
- 1.8 The deploy 15
- 1.9 What step 1 leaves behind 18

2 Stories and messages 19
- 2.1 Settings: one typed object from the environment 19
- 2.2 A database in a container 20
- 2.3 SQLAlchemy: the engine, the session, the models 20
- 2.4 Alembic: the migrations 22
- 2.5 A rule the database enforces: messages are append-only 22
- 2.6 Dependency injection, FastAPI's way 23
- 2.7 The stories API 24
- 2.8 Sign-in 25
- 2.9 Testing against a real database 26
- 2.10 What step 2 leaves behind 27

3 One turn against OpenRouter 28
- 3.1 One endpoint, one key 28
- 3.2 Streaming, in both directions 29
- 3.3 The client 30
- 3.4 The ledger 33
- 3.5 The turn 33
- 3.6 Reroll 35
- 3.7 Slash commands, and `/ask` 36

3.8 Testing a stream 37
3.9 What step 3 leaves behind 39
4 The library 40
4.1 Three shelves, one shape 40
4.2 Names 42
4.3 What a foreign key does for you 43
4.4 Optimistic concurrency 44
4.5 History 45
4.6 A note for lists, and a count for the tabs 45
4.7 Snippets expand at send time 47
4.8 What the tests look like now 48
4.9 What step 4 leaves behind 48
5 The context builder 50
5.1 Why layers, and why this order 50
5.2 The shape: dataclasses in, dataclasses out 51
5.3 Counting tokens 51
5.4 The budget 52
5.5 The audit 53
5.6 Testing pure logic 54
5.7 What step 5 leaves behind 55
6 The memory 56
6.1 One entry point 56
6.2 Summaries: what happened 57
6.3 Facts: what is true now 59
6.4 Retrieval: what was said 59
6.5 Background calls: once more, only if it could help 61
6.6 Testing the memory 61
6.7 A real run 62
6.8 What step 6 leaves behind 63
7 Story features 64
7.1 One module per feature 64
7.2 Dials: a pack of data 65
7.3 Meters: a value the model draws and the app reads back 66
7.4 Directions: turns the reader asks for without writing one 67
7.5 Regenerate with a reason 68
7.6 A model per story — built, then taken out 68
7.7 Branch, and delete from a message 69
7.8 Search, export, cost 69
7.9 Facts by hand 70
7.10 Rebuild, and purge 70
7.11 Testing eighteen features 71
7.12 What step 7 leaves behind 71
8 The SvelteKit PWA 72
8.1 Why Svelte, and why static 72
8.2 The shell: tokens, theme, and a client 73
8.3 The gate: waking, then signing in 74

8.4 The two screens the feel lives in 75
8.5 The library editor 76
8.6 Deploying a static site 77
8.7 Testing a client 77
8.8 What step 8 leaves behind 77
9 The terminal client 78
9.1 A second package in the workspace 78
9.2 Textual's model, mapped to airp's shell 79
9.3 The screens 83
9.4 Configuration and the token 84
9.5 Testing a terminal 85
9.6 What step 9 leaves behind 86
10 Persona pieces 87
10.1 The persona does not change 87
10.2 A closed list of slots 88
10.3 The joiner 89
10.4 The API, and what it reuses 90
10.5 Extracting the pieces that already exist 91
10.6 The two clients 92
10.7 Reading and editing the pool 92
10.8 Testing it 93
10.9 What this leaves behind 94

CHAPTER 0

Orientation

You already know how to build a backend. You know what a request pipeline is, what dependency injection buys you, why migrations exist, how `async` / `await` frees a thread. None of that changes. What changes is the vocabulary, the tooling, and a handful of assumptions so deep in .NET that you no longer notice them — and those are the ones that bite.

This course follows **Lustjinn** as it is built: a web app for roleplay with a memory that does not forget, the Python successor to airp. Each chapter is one step of the roadmap. Claude writes the code; the chapter explains it — what each new piece is, why it is there, and what it would be in .NET. When a chapter and the code disagree, the chapter is out of date: that is a bug.

0.1 How to read this course

Every chapter has the same shape: what we are building in this step, the code that was written, the concepts it introduced, and what changed in the repository. Four kinds of box recur:

> **THE .NET LENS**
> Maps a Python idea onto the .NET one you already know — and says where the analogy breaks.

> **WORTH KNOWING**
> Background that is useful but not needed to follow the step.

> **WATCH OUT**
> A trap. Usually one that would compile — or rather, *run* — without complaint.

> **TRY IT**
> A small exercise. Optional, but the fastest way to make a concept stick.

0.2 The map

Most of the Python toolchain has a direct counterpart in what you use every day. Keep this table nearby; later chapters assume it.

You know (.NET)	Lustjinn uses (Python)	What it is
`dotnet` CLI	**uv**	Creates projects, installs packages, runs things
nuget.org	**PyPI**	The public package registry
`.csproj`	`pyproject.toml`	The project file: name, dependencies, tool settings
`packages.lock.json`	`uv.lock`	Exact resolved versions, committed
`global.json`	`.python-version`	Pins the runtime version
Roslyn analyzers, `dotnet format`	**ruff**	Linter and formatter in one
The compiler + nullable reference types	**pyright** (strict)	The type checker
xUnit	**pytest**	Tests
A fake `HttpMessageHandler`	**httpx2's** `MockTransport`	Fakes HTTP calls in tests
Kestrel	**uvicorn**	The web server that runs the app
ASP.NET Core minimal APIs	**FastAPI**	Routing, model binding, DI, OpenAPI
`record` + DataAnnotations	**Pydantic** models	Typed data with validation
`IOptions<T>`	**pydantic-settings**	Typed configuration from the environment
EF Core	**SQLAlchemy 2.0**	ORM and query builder
EF Core migrations	**Alembic**	Schema migrations
`HttpClient`	**httpx2**	HTTP client, sync and async
`Task`, `async` / `await`	**asyncio** coroutines	Asynchronous code

The table is a starting point, not a promise. The rest of this chapter covers the four places where the mapping breaks hardest.

0.3 Break 1 — Types are checked, but not enforced

In C#, a type is a contract the compiler enforces and the runtime relies on. In Python, a type hint is an *annotation*: the interpreter stores it and otherwise ignores it.

```
def greet(name: str) -> str:
    return "Hello, " + name

greet(42)   # Runs. Fails only inside, at the "+", with a TypeError.
```

Nothing stops that call at run time. What stops it is **pyright**, run before the code ships — exactly the role the C# compiler plays, except that it is a separate tool and you could skip it. Lustjinn runs pyright in **strict** mode, in CI, on every push: that is our `TreatWarningsAsErrors`.

THE .NET LENS

`str | None` is Python's `string?`. With nullable reference types on, C# warns when you dereference something that might be null; pyright strict does the same for `None`. The difference is that C#'s `null` checks are warnings you opted into, while in strict pyright they are errors from the start.

WORTH KNOWING

Some libraries *do* read type hints at run time, on purpose. **Pydantic** and **FastAPI** are the two you will meet first: a parameter declared `limit: int` makes FastAPI parse and validate the query string as an integer and return a 422 if it is not. That is model binding, driven by the same annotations pyright checks.

0.4 Break 2 — There is no build step

`dotnet build` turns your project into an assembly; `dotnet run` runs the assembly. Python has no equivalent step you run yourself. The interpreter (**CPython**, the standard one) compiles each file to bytecode the first time it is imported, caches it in a `__pycache__` folder, and executes it. A syntax error surfaces when the file is loaded; a typo in a name surfaces only when that line runs.

That is why the tooling matters more than in .NET: **ruff** and **pyright** are the build. If they are clean and the tests pass, the code is as checked as it will get before it runs.

0.5 Break 3 — Every project gets its own interpreter environment

NuGet keeps packages in a global cache, and each project references the versions it wants; two projects can use different versions of the same package without noticing each other.

Python installs packages *into an interpreter*. Install a package into the system Python and every project on the machine sees that version. The answer is a **virtual environment**: a folder (`.venv`) holding a private copy of the interpreter's package directory, one per project.

uv manages all of it. It downloads the Python version the project pins, creates `.venv`, installs exactly what `uv.lock` says, and runs commands inside that environment:

```
uv sync              # make .venv match uv.lock          ≈ dotnet restore
uv add httpx2        # add a dependency, update the lock  ≈ dotnet add package
uv run pytest        # run a command inside .venv         ≈ dotnet test
uv python install    # fetch the pinned Python version    ≈ installing an SDK
```

WATCH OUT

Your Windows machine has an older system Python (3.9) on the `PATH`. Lustjinn never uses it: uv fetches the version `.python-version` asks for and keeps it separate. If a command ever behaves as if it were on 3.9, it was run with `python` instead of `uv run python`.

0.6 Break 4 — Async runs on one thread

In ASP.NET Core, each request runs on a thread-pool thread. If a handler blocks — a synchronous database call, a `Thread.Sleep` — it wastes that one thread, and the pool covers for it.

Python's `asyncio` runs every coroutine on **a single thread**, inside an **event loop**. An `await` hands control back to the loop so another request can make progress. A call that blocks without awaiting does not waste one thread; it freezes *every* request the server is handling until it returns.

```
async def handler():
    time.sleep(2)           # Blocks the event loop: the whole server stops for 2 s.
    await asyncio.sleep(2)  # Yields to the loop: other requests keep running.
```

This is why Lustjinn uses the async versions of everything that waits: **httpx2**'s `AsyncClient`, SQLAlchemy's async engine, an async Postgres driver.

> **THE .NET LENS**
>
> `async def` ≈ an `async Task` method, and `await` means the same thing. The difference is what happens *between* awaits: in .NET, other work runs on other threads; in Python, other work runs only when you await. There is no `ConfigureAwait`, and no `SynchronizationContext` to think about.

> **WORTH KNOWING**
>
> FastAPI hides one sharp edge for you. An endpoint declared with plain `def` (not `async def`) is run on a separate thread pool, so blocking inside it is safe. An endpoint declared `async def` runs on the event loop and must never block.

0.7 Smaller differences you will meet soon

- **Files are modules; folders are packages.** There is no `namespace` keyword and no project reference: `from lustjinn.stories import Story` imports by path from the package folder.
- **Naming:** `snake_case` for functions, variables and modules; `PascalCase` for classes; `UPPER_CASE` for constants. The leading underscore (`_helper`) means "private by convention" — nothing enforces it.
- **`self` is explicit.** A method's first parameter is the instance, written out. The constructor is `__init__`.
- **Indentation is syntax.** Blocks have no braces; the indentation *is* the block. ruff's formatter keeps it consistent.
- **`None`, `True`, `False`** are capitalised.

0.8 What is already on your machine

The setup for this course and for step 1 is installed:

- **uv** — the Python toolchain. It will fetch the current Python on first use.
- **pandoc** and **Typst** — they turn the Markdown chapters of this course into this PDF.

The course lives in the repository under `docs/course/`. To rebuild it:

```
./docs/course/build.ps1          # the dark edition
./docs/course/build.ps1 -Light   # a light edition, for printing
```

> **TRY IT**
>
> Run `uv --version` and `uv python list`. The second command lists the Python versions uv can install, and any it already has. Next chapter, one of them becomes Lustjinn's.

0.9 What is next

Chapter 1 — Hello, deployed. A uv project, FastAPI with one endpoint, one pytest test, ruff and pyright in CI, and the whole thing deployed to Render — so that `GET /health` answers from the internet. It teaches the project layout, uv in practice, and how a Python web app is actually served.

CHAPTER 1

Hello, deployed

The goal of step 1 is the smallest thing that is real: a Python web app with one endpoint, one test, the quality gates, and a path to a public URL. Nothing about roleplay yet. Everything later grows out of this skeleton, so it is worth understanding every file in it.

The step landed in three pieces, and this chapter follows them in order:

1. **The project** — uv, FastAPI, one endpoint, one test, ruff and pyright.
2. **CI** — the gates running on GitHub Actions on every pull request.
3. **The deploy** — the path to Render, built and waiting: the project runs locally until it is ready for the cloud.

1.1 What uv created

One command started the project:

```
uv init --package --name lustjinn --python 3.14
```

and two more added the libraries:

```
uv add fastapi "uvicorn[standard]"
uv add --dev pytest httpx2 ruff pyright
```

The result, ignoring the docs:

```
.python-version          3.14 — which Python this project runs on
pyproject.toml           the project file
uv.lock                  every package, pinned to an exact version and hash
.venv/                   the project's private environment (gitignored)
src/lustjinn/
    __init__.py          makes the folder a package
    main.py              the FastAPI app
tests/
    test_health.py       the one test
```

Python 3.14 is pinned because it is the newest stable release: 3.15 exists, but only as a release candidate. uv downloaded 3.14.7 on its own, without touching the Python 3.9 already installed on the machine.

1.1.1 Why the code lives under `src/`

The package is `src/lustjinn/`, not `lustjinn/` at the root. With the code at the root, `import lustjinn` would work in tests simply because the current folder happens to be on the import path — and a test could pass against files that would never be part of the installed app. Under `src/`, the only way to import `lustjinn` is to install it, which uv does: `uv sync` installs the project into `.venv` in *editable* mode, meaning the environment points at `src/` instead of copying it, so edits are live.

> **THE .NET LENS**
>
> This is the test project's `<ProjectReference>`. A test project in .NET cannot see the app's source files, only the assembly the reference builds. The `src/` layout gives Python the same discipline: tests see the package as installed, the way the server will.

1.2 The project file

`pyproject.toml` is the `.csproj`. It is TOML, a format of `[tables]` and `key = value` lines.

```toml
[project]
name = "lustjinn"
version = "0.1.0"
requires-python = ">=3.14"
dependencies = [
    "fastapi>=0.142.2",
    "uvicorn[standard]>=0.54.0",
]

[dependency-groups]
dev = [
    "httpx2>=2.13.1",
    "pyright>=1.1.414",
    "pytest>=9.1.1",
    "ruff>=0.16.10",
]
```

- **`[project]`** is the standard part every Python tool understands: name, version, the Python it needs, and the packages the app needs at run time.
- **`uvicorn[standard]`** — the brackets name an *extra*: optional dependencies a package offers. `standard` adds uvicorn's faster event loop and HTTP parser, and file watching for `--reload`.
- **`[dependency-groups]`** holds what development needs but production does not: the test runner and the gates. A production install leaves them out.
- **`[build-system]`** says how to turn the project into an installable package (`uv_build`).
- **`[tool.…]`** tables configure each tool — `[tool.ruff]`, `[tool.pyright]`, `[tool.pytest…]` — all in the one file.

> **THE .NET LENS**
>
> `"fastapi>=0.142.2"` reads like NuGet's `Version="0.142.2"`, and both mean *at least* that version. The difference is which version gets picked: NuGet resolves the **lowest** version that satisfies every constraint, uv the **highest**. Either way, the lock file is what makes the build repeatable — `uv.lock` plays the role of `packages.lock.json`, and it is committed.

1.3 The app

The first version of the whole application, `src/lustjinn/main.py` (the deploy, at the end of the chapter, grows it by one field):

```python
"""The FastAPI application: the object uvicorn serves."""

from fastapi import FastAPI

app = FastAPI(title="Lustjinn")


@app.get("/health")
async def health() -> dict[str, str]:
    """Answers while the process is up. Render and the PWA's waking screen poll it."""
    return {"status": "ok"}
```

Line by line:

- The string at the top of a file, function or class is a **docstring**: documentation the language keeps (`help()` shows it). It plays the part of an XML `<summary>` comment.
- `from fastapi import FastAPI` imports one name from a package — a `using` that names exactly what it brings in.
- `app = FastAPI(...)` creates the application at module level. When Python imports this file, the line runs, and `app` exists from then on.
- `@app.get("/health")` is a **decorator**. Python calls `app.get("/health")`, which returns a function; that function receives `health`, registers it as the handler for `GET /health`, and hands it back unchanged. The `@` line is shorthand for exactly that.
- `async def health() -> dict[str, str]` declares a coroutine that returns a dictionary of strings. FastAPI reads the return annotation: it documents the response in the OpenAPI schema and validates what the function returns.
- `return {"status": "ok"}` — FastAPI serialises the dictionary to JSON.

> **THE .NET LENS**
>
> Put side by side with a minimal API, there is almost nothing to translate:
>
> ```
> var app = WebApplication.CreateBuilder(args).Build();
> app.MapGet("/health", () => new Dictionary<string, string> { ["status"] = "ok" });
> app.Run();
> ```
>
> `app.MapGet(...)` becomes the decorator, and `Dictionary<string, string>` becomes `dict[str, str]`. What has no counterpart is `app.Run()` — see the next section.

1.4 How Python is served

A .NET app is an executable: `Program.cs` builds the app and `app.Run()` starts Kestrel inside the same process. A Python web app is not run; it is **imported by a server**. The server here is **uvicorn**:

```
uv run uvicorn lustjinn.main:app --reload
```

`lustjinn.main:app` means *import the module* `lustjinn.main` *and serve the object called* `app` *in it.* uvicorn owns the process, the socket and the event loop; FastAPI only answers requests handed to it. `--reload` restarts the server when a file changes — for development only.

The contract between the two is **ASGI** (Asynchronous Server Gateway Interface): a standard signature every async Python server can call and every async framework implements. That is why FastAPI could run on another ASGI server without changing a line.

> **THE .NET LENS**
> uvicorn is Kestrel, and ASGI is the boundary between Kestrel and the ASP.NET Core pipeline — except that in Python the boundary is a public standard, so servers and frameworks mix and match.

With the server running, FastAPI also serves its own documentation: `/docs` is an interactive Swagger UI and `/openapi.json` the schema, both generated from the code — no Swashbuckle to add.

1.5 The test

The first version of `tests/test_health.py`:

```python
from fastapi.testclient import TestClient

from lustjinn.main import app

client = TestClient(app)


def test_health_answers_ok() -> None:
    response = client.get("/health")

    assert response.status_code == 200
    assert response.json() == {"status": "ok"}
```

- **pytest finds tests by name**: files called `test_*.py`, functions called `test_*`. No attribute, no test class needed.
- **Plain `assert`.** pytest rewrites assert statements when it loads a test file, so a failure shows both sides of the comparison — no `Assert.Equal` family to learn.
- `TestClient` runs the app in-process and speaks to it over ASGI directly: no server, no port, no network.

> **THE .NET LENS**
> `TestClient(app)` is `WebApplicationFactory<Program>` plus `CreateClient()`, in one line.

> **WATCH OUT**
> When this test was first written, the dev dependency was `httpx`, as countless tutorials say. The test passed — with a deprecation warning, because Starlette (the toolkit under FastAPI) now wants its successor, `httpx2`. pyright went further and failed: with `httpx2` missing, it could not know the types of `client.get`, `response.status_code` or `response.json`, and strict mode refuses unknown types. Two lessons in one. Tutorials age fast in Python — check what the

> library itself asks for. And strict typing earns its keep on day one: it turned a quiet warning into an error.

1.6 The gates

Four commands decide whether a change is acceptable. All four are clean on this commit:

```
uv run pytest               # the tests
uv run ruff check           # lint
uv run ruff format --check  # formatting, without rewriting
uv run pyright              # types, strict
```

ruff is configured in `pyproject.toml` with a deliberate set of rule families. Most are style and likely-bug checks. One deserves a mention: `ASYNC` flags blocking calls made inside `async def` functions — the event-loop freeze from chapter 0, caught before it runs.

pyright runs in `strict` mode over `src` and `tests`. Strict means every value must have a type pyright can know; an unknown type is an error, not a shrug.

> **TRY IT**
>
> Start the server with `uv run uvicorn lustjinn.main:app --reload` and open `http://127.0.0.1:8000/docs`. Call `/health` from the Swagger page.
>
> Then, in `main.py`, change the return to `{"status": 1}` and run `uv run pyright`. The annotation says `dict[str, str]`; pyright says no. Change it back.

1.7 CI on GitHub Actions

The gates are only worth something if nothing reaches `develop` without passing them. A workflow in `.github/workflows/ci.yml` runs all four on GitHub, where the repository lives:

```yaml
on:
  push:
    branches: [main]
  pull_request:
    branches: [main, develop]

concurrency:
  group: ci-${{ github.event.pull_request.number || github.run_id }}
  cancel-in-progress: ${{ github.event_name == 'pull_request' }}

jobs:
  gates:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    env:
      UV_VERSION: "0.12.22"
    steps:
      - uses: actions/checkout@v5
        with:
          persist-credentials: false
      - name: Install uv
```

```
        run: |
          curl -LsSf "https://astral.sh/uv/${UV_VERSION}/install.sh" | sh
          echo "$HOME/.local/bin" >> "$GITHUB_PATH"
      - name: Sync the environment from uv.lock
        run: uv sync --locked
      - name: ruff check
        run: uv run ruff check
      - name: ruff format
        run: uv run ruff format --check
      - name: pyright (strict)
        run: uv run pyright
      - name: pytest
        run: uv run pytest
```

Reading it top to bottom:

- `on` — a pull request into `develop` or `main` runs the gates. A push to `develop` does **not**: `develop` only moves by merging a pull request whose gates already ran, so running them again would test the same code twice. A push to `main` — a release — does.
- `concurrency` — a new push to a pull request cancels that pull request's previous run. Only the newest commit's result matters.
- `runs-on: ubuntu-latest` — a fresh Linux virtual machine for every run. uv and the project's Python come from the steps below, so a green run proves the project builds from its committed files alone.
- `timeout-minutes` — a stuck job stops after ten minutes. The default is six hours.
- **Install uv** — one pinned release, the same version as on the development machine. Python then comes from uv, exactly as on Windows: `uv sync` reads `.python-version` and fetches 3.14.
- `uv sync --locked` — creates the environment from `uv.lock`, and **fails** if the lock no longer matches `pyproject.toml`. Without `--locked`, uv would quietly re-resolve, and CI would test versions nobody committed.
- **The gates, cheapest first** — a formatting slip fails in seconds, before pyright and the tests run.

> **THE .NET LENS**
>
> `uv sync --locked` is `dotnet restore --locked-mode`: restore exactly what the lock file says, or fail. And the workflow as a whole is the pipeline you would write around `dotnet build` and `dotnet test` — with `uv sync` as the restore and pyright and ruff standing in for the build.

> **WORTH KNOWING**
>
> The repository is private, so every minute a workflow runs is billed against the account's monthly allowance, and each job is rounded up to a whole minute. That is the reason for three of the choices above: nothing runs on a `develop` push, superseded runs are cancelled, and every job has a timeout. A run of these gates takes under a minute.

1.8 The deploy

The last piece is the path to the internet: a **Render** free web service, called `lustjinn-staging`, serving the `develop` branch. The path is built and tested as far as it can be without an account — but it has not run yet. The project stays on the development machine until it is worth putting in the cloud, and this section describes what will happen then.

1.8.1 What Render runs

Render's native Python runtime needs no Dockerfile. It reads `.python-version` to choose the interpreter, and because the repository contains `uv.lock`, it installs uv too. The service is described in `render.yaml` at the root of the repository — a **blueprint**:

```yaml
services:
  - type: web
    name: lustjinn-staging
    runtime: python
    plan: free
    branch: develop
    buildCommand: uv sync --locked --no-dev
    startCommand: uv run --no-sync uvicorn lustjinn.main:app --host 0.0.0.0 --port $PORT
    healthCheckPath: /health
    autoDeployTrigger: "off"
    envVars:
      - key: UV_VERSION
        value: "0.12.22"
```

- **The build** is the same `uv sync --locked` as CI, plus `--no-dev`: pytest, ruff and pyright stay out of production.
- **The start** runs uvicorn through `uv run --no-sync` — inside the environment the build created, without checking it again. It binds to `0.0.0.0`, every network interface, because Render's proxy reaches the process from outside the container; the default, `127.0.0.1`, would accept only connections from inside. And it listens on `$PORT`, the port Render chooses.
- `healthCheckPath` — Render calls `/health` to decide whether the service is up.
- `autoDeployTrigger: "off"` — a push alone deploys nothing; a deploy is always asked for.

> **THE .NET LENS**
>
> `--host 0.0.0.0 --port $PORT` is Kestrel's `ASPNETCORE_URLS=http://+:$PORT` — the same lesson every container platform teaches: listen where the platform says, on every interface. And the blueprint is infrastructure as code, a very small Bicep file: the service lives in the repository, not only in a dashboard.

> **WATCH OUT**
>
> The free tier has two properties the whole design respects. The service **sleeps** after about fifteen minutes without requests and takes 30–60 seconds to wake — which is why the future PWA is a separate static site that opens instantly and shows "waking the server". And it has **no persistent disk**: anything written to the filesystem disappears on the next deploy or restart. No SQLite, no files. Everything lives in the database: a local Postgres for now, Neon once the app moves to the cloud.

1.8.2 How a deploy travels

A deploy is started by hand: on GitHub, *Actions → CI → Run workflow* on `develop`, with **deploy = staging**. The same workflow that runs the gates then runs one more job, `deploy-staging`, only if they all passed:

1. **Fire the hook.** A `POST` to Render's *deploy hook* — a secret URL — starts a build of the branch.

2. **Wait for the right commit.** The job polls `/health` until the service reports *this* commit. "The service is up" is not enough: the old version is up too, until the new one replaces it.

The one-time setup — creating the service from the blueprint, the hook secret and the service URL on GitHub — is in `docs/DEPLOYMENT.md`.

1.8.3 `/health` grows a field

Step 2 of that list needs the service to say which commit it runs. Render provides it in an environment variable, `RENDER_GIT_COMMIT`, so `/health` now returns it:

```python
import os
from typing import Literal

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="Lustjinn")


class Health(BaseModel):
    status: Literal["ok"]
    commit: str | None
    """The deployed commit (Render's `RENDER_GIT_COMMIT`); None when not running on Render."""


@app.get("/health")
async def health() -> Health:
    """Answers while the process is up. Render, the deploy job and the PWA poll it."""
    return Health(status="ok", commit=os.environ.get("RENDER_GIT_COMMIT"))
```

This is the first **Pydantic model**. A class that inherits `BaseModel` declares its fields as annotated class attributes; Pydantic validates them when an instance is created and serialises them to JSON. FastAPI shows the model in the OpenAPI schema under its own name, `Health`.

- `Literal["ok"]` — a type with exactly one allowed value. `Health(status="fine")` fails validation, and pyright rejects it before it runs.
- `str | None` — a string or nothing. `os.environ.get(...)` returns `None` when the variable is not set, so off Render the answer is `{"status": "ok", "commit": null}`.

> **THE .NET LENS**
>
> `Health` is a `record Health(string Status, string? Commit)` with validation attached. `os.environ.get` is `Environment.GetEnvironmentVariable`, which also returns null when the variable is missing. `Literal["ok"]` has no direct C# counterpart; the nearest is a one-value enum.

The tests grow with it:

```python
def test_health_reports_the_deployed_commit(monkeypatch: pytest.MonkeyPatch) -> None:
    monkeypatch.setenv("RENDER_GIT_COMMIT", "3d36977")

    assert client.get("/health").json()["commit"] == "3d36977"


def test_health_reports_no_commit_off_render(monkeypatch: pytest.MonkeyPatch) -> None:
    monkeypatch.delenv("RENDER_GIT_COMMIT", raising=False)
```

```
    assert client.get("/health").json() == {"status": "ok", "commit": None}
```

`monkeypatch` is a **fixture**: pytest sees a parameter with that name and passes in a ready-made object. `setenv` and `delenv` change the environment for this test only — pytest undoes both when it ends, so no test leaks state into the next.

> **THE .NET LENS**
>
> Fixtures are pytest's dependency injection — by parameter name, the way ASP.NET Core injects by type. xUnit's nearest equivalents are constructor injection and `IClassFixture<T>`; the automatic undo is what you would otherwise write in `Dispose()`.

> **TRY IT**
>
> Run the server with a commit of your own, in PowerShell:
>
> ```
> $env:RENDER_GIT_COMMIT = "hello"; uv run uvicorn lustjinn.main:app
> ```
>
> `/health` now reports `"commit": "hello"`, and `/docs` shows the `Health` schema. Then try `Health(status="fine", commit=None)` in `main.py` and run `uv run pyright`.

1.9 What step 1 leaves behind

- A Python 3.14 project managed by uv, locked, with a `src/` layout.
- A FastAPI app with one endpoint, three tests, and four gates — all clean.
- CI on GitHub Actions running the gates on every pull request.
- A deploy path to Render staging that refuses to run on red gates and checks what it deployed — ready for the day the app moves to the cloud.

Next — Chapter 2, Stories and messages. The first real data, in a Postgres running in Docker: settings from the environment, sign-in, SQLAlchemy models, the first Alembic migration, and a trigger that makes messages append-only.

CHAPTER 2

Stories and messages

Step 1 built an app that could say it was alive. Step 2 gives it something to keep: a database, the tables for stories and their messages, a rule the database enforces by itself, an API over them, and a lock on the door. Everything runs on your machine — Postgres in a Docker container, the API under uvicorn. The cloud comes later and changes one connection string.

This is the chapter where Python's backend stack stops looking like a toy and starts looking like what you already know. Almost every piece has a .NET twin; the interesting parts are the few places where the twin behaves differently.

2.1 Settings: one typed object from the environment

Before anything can connect to anything, the app needs to be told where the database is and who may sign in. `src/lustjinn/settings.py`:

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_prefix="LUSTJINN_", env_file=".env", extra="ignore")

    database_url: str
    username: str
    password: SecretStr
    token_secret: SecretStr = Field(min_length=32)
    token_days: int = Field(default=30, ge=1, le=365)
    cors_origins: list[str] = []


@lru_cache
def get_settings() -> Settings:
    return Settings()
```

`BaseSettings` is a Pydantic model that fills itself in: each field is read from the environment variable of the same name with the prefix — `LUSTJINN_DATABASE_URL`, `LUSTJINN_USERNAME` — or from a `.env` file in the working directory, real variables winning. A field with no default is required. Types are enforced: `token_days` must parse as an integer between 1 and 365.

- **`SecretStr`** wraps a string so that printing or logging the settings shows `**********`. You get the value out on purpose, with `.get_secret_value()`.
- **`@lru_cache`** on a function with no arguments turns it into "compute once, then return the same object": the settings are read on the first call and reused after that.

- **Failing early.** The app's *lifespan* — code FastAPI runs once at start-up and once at shutdown — calls `get_settings()`. A missing password stops uvicorn before it accepts a single request, with an error naming the field.

> **THE .NET LENS**
>
> `Settings` is the class you would bind with `services.Configure<Settings>(config)` and receive as `IOptions<Settings>`; the environment-variable and `.env` layering is the configuration builder; `Field(min_length=32)` is a DataAnnotation plus `ValidateOnStart()`. The lifespan is `IHostedService.StartAsync` / `StopAsync` in one function, split by a `yield`.

`.env` is gitignored. `.env.example` is committed and lists every variable; copy it and fill in the blanks.

2.2 A database in a container

`docker-compose.yml` describes one service: Postgres 17 with the pgvector extension available, listening on port 5440 of this machine only, its data in a named volume.

```
docker compose up -d      # start it; the data survives restarts
docker compose down       # stop it; add -v to throw the data away
```

Port 5440 rather than the usual 5432 so it never collides with another project's Postgres. Postgres 17 because Neon offers it: when the app moves to the cloud, the schema and every query stay as they are.

2.3 SQLAlchemy: the engine, the session, the models

SQLAlchemy is the ORM. Three objects carry it, and each has an EF Core counterpart — though the unit of work is more explicit here.

2.3.1 The engine and the session

```python
@lru_cache
def get_engine() -> AsyncEngine:
    return create_async_engine(get_settings().database_url, pool_pre_ping=True)


@lru_cache
def get_sessionmaker() -> async_sessionmaker[AsyncSession]:
    return async_sessionmaker(get_engine(), expire_on_commit=False)


async def get_session() -> AsyncGenerator[AsyncSession]:
    async with get_sessionmaker()() as session:
        yield session
```

- The **engine** owns the connection pool. One per process, created on first use.
- A **session** is one conversation with the database: it tracks the objects you loaded or added, and writes the changes when told. One per request.
- `get_session` is a generator: it yields a session to whoever asked, and the code after the `yield` — here, leaving the `async with`, which closes the session — runs when the request ends.

THE .NET LENS

The engine is the pooled connection factory behind `AddDbContext`. The session is the `DbContext`: change tracking, identity map, one unit of work, scoped to a request. Two differences will matter all through this project:

- **`flush()` and `commit()` are separate.** `flush()` sends the pending INSERTs and UPDATEs but leaves the transaction open; `commit()` ends it. EF's `SaveChanges()` does both.
- **There is no lazy loading in async code.** Touching `story.messages` cannot quietly run a query, because a query needs an `await`. The relationships are declared `lazy="raise"`, so a forgotten load fails loudly instead of deadlocking or lying. You load what you need, in the query — the discipline EF asks for with `Include`, enforced.

`expire_on_commit=False` is the other async accommodation: by default SQLAlchemy forgets an object's values at commit and reloads them on next access, which again would be a hidden query.

2.3.2 The models

`src/lustjinn/models.py`, abridged:

```
class Story(Base):
    __tablename__ = "stories"

    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=new_id)
    name: Mapped[str] = mapped_column(String(200))
    character_id: Mapped[uuid.UUID] = mapped_column(
        ForeignKey("characters.id", ondelete="RESTRICT"), index=True
    )
    persona_id: Mapped[uuid.UUID | None] = mapped_column(
        ForeignKey("personas.id", ondelete="RESTRICT"), index=True
    )
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
server_default=func.now())
    deleted_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
```

A model is a class whose annotated attributes are columns. `Mapped[str]` is a NOT NULL text column; `Mapped[str | None]` is nullable — the type hint *is* the nullability, checked by pyright and enforced by Postgres. `mapped_column(...)` adds what the type cannot say: lengths, keys, defaults, indexes.

Four tables exist after this step: `characters` and `personas` (minimal for now; the library step fills them out), `stories`, and `messages`. The decisions worth noticing:

- **Ids are UUID version 7** (`uuid.uuid7()`, new in Python 3.14): unique like any UUID, but beginning with a timestamp, so they sort by creation time and index well.
- **A story points at its character by id**, with `ON DELETE RESTRICT`. airp stored the character's *name*, so renaming one was impossible, and "not while a story uses it" was checked in code. Here the database refuses the delete itself.
- **Deleting is hiding.** `deleted_at` is set and every query filters on it. The row stays.
- **A message has a `sequence`**, unique within its story, and a `request_hash` that is unique within its story *among rows that have one* — a **partial index**. Step 3 uses the hash to make a retried request harmless.
- **`model` is null when a person wrote the message.** The reader's turns, and a story's opening. A later step reads that null to know an opening must not be rerolled.

> **THE .NET LENS**
>
> `Mapped[...]` and `mapped_column` are the entity class and its `OnModelCreating` configuration merged into one declaration. `ForeignKey(..., ondelete="RESTRICT")` is `.OnDelete(DeleteBehavior.Restrict)`. The partial index is `HasIndex(...).HasFilter("...")`.

2.4 Alembic: the migrations

The models say what the schema should be. **Alembic** gets a database there, one versioned step at a time, and records in a table (`alembic_version`) how far it has gone.

```
uv run alembic revision --autogenerate -m "what changes"   # write a migration from the models
uv run alembic upgrade head                                # apply what is not applied yet
uv run alembic downgrade -1                                # undo the last one
```

Autogenerate compares the models with a live database and writes the difference as Python: `op.create_table(...)`, `op.create_index(...)`. The result is a draft — it is read, tidied and committed like any code. `migrations/versions/0001_stories_and_messages.py` began that way.

`migrations/env.py` is the file Alembic runs for every command. Ours does two things: it takes the database URL from `get_settings()`, so there is one source of truth, and it accepts a connection handed in from outside — which is how the tests run the real migrations against their own database.

> **THE .NET LENS**
>
> `alembic revision --autogenerate` is `dotnet ef migrations add`; `alembic upgrade head` is `dotnet ef database update`; `alembic_version` is `__EFMigrationsHistory`. The difference: there is no model snapshot file. Alembic compares against a real database every time, so the database must be at the previous migration when you generate the next.

One test keeps the two descriptions honest: it applies every migration to an empty database and asks Alembic whether anything still differs from the models. A model edited without a migration fails the build.

2.5 A rule the database enforces: messages are append-only

A story's memory is derived from its transcript — summaries, facts, embeddings. If a message could be edited or deleted, everything built from it would silently become a lie. airp guarded this in `SaveChanges`. That protects against the application; it does nothing about a script, a console session, or a second client. So here the rule lives where the data lives.

Migration `0002` is hand-written SQL, because a trigger is not something a model can express:

```
CREATE FUNCTION messages_append_only() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'DELETE' THEN
        IF current_setting('lustjinn.purging', true) = 'on' THEN
            RETURN OLD;
        END IF;
        RAISE EXCEPTION 'messages are append-only: a message is hidden with deleted_at, ...'
```

```
            USING ERRCODE = 'restrict_violation';
    END IF;

    IF NEW.text IS DISTINCT FROM OLD.text
       OR NEW.role IS DISTINCT FROM OLD.role
       OR NEW.story_id IS DISTINCT FROM OLD.story_id
       OR NEW.sequence IS DISTINCT FROM OLD.sequence THEN
        RAISE EXCEPTION 'messages are append-only: text, role, story and sequence cannot ...'
            USING ERRCODE = 'restrict_violation';
    END IF;
    RETURN NEW;
END
$$;

CREATE TRIGGER messages_append_only
BEFORE UPDATE OR DELETE ON messages
FOR EACH ROW EXECUTE FUNCTION messages_append_only();
```

A trigger function runs for each affected row, before the change. `OLD` and `NEW` are the row before and after; `RAISE EXCEPTION` aborts the statement. Reading it:

- **A delete is refused** — unless the transaction has announced a purge with `SET LOCAL lustjinn.purging = 'on'`. That is the one door, for the day a story is erased for good, and it closes by itself when the transaction ends.
- **What a turn said, who said it and where it sits cannot change** — purge or no purge.
- **Everything else may**: `deleted_at`, to hide a message; `request_hash`; columns later steps add.
- A second trigger refuses `TRUNCATE`, which empties a table without visiting rows and so never fires the first.

The error code is one of Postgres's own integrity codes, so SQLAlchemy raises `IntegrityError` — the same exception as for a broken foreign key. To the application the trigger is one more constraint.

> **WORTH KNOWING**
>
> `IS DISTINCT FROM` is `<>` that treats NULL as a value: `NULL <> NULL` is unknown, while `NULL IS DISTINCT FROM NULL` is false. In a guard you want the second.

2.6 Dependency injection, FastAPI's way

Every endpoint that touches the database needs a session. It asks for one in its signature:

```
# src/lustjinn/deps.py
Session = Annotated[AsyncSession, Depends(get_session)]


# src/lustjinn/stories.py
@router.get("")
async def list_stories(session: Session) -> list[StoryOut]:
    rows = await session.execute(_stories())
    return [StoryOut.model_validate(row) for row in rows]
```

`Depends(get_session)` tells FastAPI: before calling this function, call `get_session` and pass in what it yields. `Annotated[X, ...]` attaches that instruction to the type, and the alias `Session` gives the pair a name so every endpoint can write `session: Session`.

The aliases live in one module, `deps.py` — the session, the settings (`CurrentSettings`) and the model client (`Model`) — and every route module imports them. For a while each module declared its own `Session`, thirteen copies of one line; the day injection changes, that is thirteen places to find (#129).

There is no container and no registration step. A dependency is any callable; its own parameters can be dependencies in turn; a generator dependency gets its cleanup run after the response.

> **THE .NET LENS**
>
> `Depends(get_session)` is constructor injection of a scoped service, declared at the point of use instead of in `Program.cs`. The lifetime is per request, like `AddScoped`. What replaces `ConfigureTestServices` is `app.dependency_overrides[get_session] = something_else` — a dictionary from the real provider to its stand-in. The tests use exactly that.

There is no repository between the endpoint and the session, and no service layer: an endpoint writes its `select(...)` where it needs it. That is a decision, not an omission, and it is written down in `docs/adr/0001-data-access.md` (#134) — one user, tests against a real Postgres, and `dependency_overrides` already giving the test seam a repository would have — with the condition that would reverse it: the day there is a second user, every query needs an owner, and a layer that scopes them all becomes worth its file.

2.7 The stories API

`src/lustjinn/stories.py` holds five endpoints under `/stories`: create, list, read one with its messages, rename, delete. Three things in it are new.

Pydantic models are the DTOs. What comes in and what goes out are declared as classes, and FastAPI validates the first and serialises the second:

```
Name = Annotated[str, StringConstraints(strip_whitespace=True, min_length=1, max_length=200)]


class NewStory(BaseModel):
    name: Name
    character_id: uuid.UUID
    persona_id: uuid.UUID | None = None
```

A body with a blank name, or an id that is not a UUID, never reaches the function: FastAPI answers 422 with the field and the reason.

A response is read off what the database returns. `StoryOut` says `model_config = ConfigDict(from_attributes=True)`, so `StoryOut.model_validate(row)` builds it by reading the attributes of the same names. The query that feeds it labels its columns to match — `Character.name.label("character_name")`, `func.left(newest.c.text, 200).label( "last_message_preview")` — so a row *is* a story as the API shows it, and no hand-written mapper copies field after field (#130). A new column is then one line in the query and one in the model, not a third in a mapper that is easy to forget.

> **THE .NET LENS**

> `from_attributes` is AutoMapper's convention mapping, or a hand-written `Select(r => new StoryOut { ... })` that EF Core translates — except that the projection here is the SQL's own column names, and Pydantic only reads them. `.label(...)` is `AS character_name`.

A router groups endpoints. `APIRouter(prefix="/stories")` is `MapGroup("/stories")`; the app includes it with `app.include_router(...)`.

The list is one query. For each story it needs the newest visible message — its time, to sort by, and its first 200 characters, to show under the name. That is a `LATERAL` subquery: one that runs per outer row and may refer to it. EF Core produces the same SQL from a correlated `OrderByDescending(...).FirstOrDefault()`; here it is written out.

Creating a story does one more thing: if its character has an **opening**, the opening becomes message 1, role `assistant`, with no model — a page a person wrote, not a reply to regenerate.

2.8 Sign-in

The API will one day sit on a public URL with one user's private stories behind it. `src/lustjinn/auth.py`:

1. `POST /auth/sign-in` takes the username and password and compares them with the settings.
2. If they match, it returns a **token**: a small signed document saying who it is for and when it expires (30 days by default).
3. Every other request carries `Authorization: Bearer <token>`. A dependency, `require_user`, checks the signature and the expiry before the endpoint runs.

The token is a **JWT** signed with HMAC-SHA256 using `LUSTJINN_TOKEN_SECRET`. Nothing is stored on the server: the signature proves the token was issued here, and the expiry is inside it. Changing the secret invalidates every token at once — that is "sign out everywhere".

The guard is attached where the router is included, not on each endpoint:

```
app.include_router(auth.router)
# Everything below needs a valid token.
app.include_router(stories.router, dependencies=[Depends(auth.require_user)])
```

> **THE .NET LENS**
> This is `AddAuthentication().AddJwtBearer()` plus `MapGroup("/stories").RequireAuthorization()`, without the middleware: the check is an ordinary dependency. `HTTPBearer` is the piece that reads the header and puts an *Authorize* button on the `/docs` page.

Details that are easy to get wrong, each pinned by a test:

- **Comparison in constant time** (`secrets.compare_digest`), and both name and password are checked before answering, so neither the reply nor its timing says which one was wrong.
- **The accepted algorithm is named explicitly.** A forged token whose header says "no signature needed" is refused because the server never asks the token how to verify it.
- **A token without an expiry is refused**, as is one issued for a different username.

- **One test covers every route**: it reads the app's own OpenAPI description and requires a 401 from each path except `/health` and `/auth/sign-in`. An endpoint added next month is guarded, or that test fails.

> **WATCH OUT**
> FastAPI's default 422 response includes the input it rejected. For a malformed sign-in that input is the password; for a story it is the text. A test caught the first, and the app now answers validation errors with the field and the problem only. Defaults are written for tutorials — read what yours actually send.

Two more things ride along. **CORS** — which browser origins may call the API — comes from the settings and allows none by default; the web client's origin gets added when it exists. And every response carries four headers saying it is private: not to be cached, indexed, or referred from.

2.9 Testing against a real database

The trigger, the constraints and the queries are Postgres behaviour. Faking the database would test the fake. So the tests use the real thing — and never the data you are playing with.

`tests/conftest.py` builds it from **fixtures**, each asking for the one before by naming it as a parameter:

1. `database_url` (once per run) creates an empty database with a random name on the same server, and drops it at the end.
2. `engine` (once per run) applies every Alembic migration to it — the real migrations, so they are tested on every run.
3. `session` (per test) opens a transaction that is **never committed**, and rolls it back when the test ends. The code under test may call `commit()` freely: the session turns those into savepoints inside the outer transaction.
4. `client` (per test) is an HTTP client wired straight to the app in-process, with the app's `get_session` and `get_settings` overridden to hand out the test's own.

```python
async def test_deleting_a_story_hides_it_and_keeps_its_rows(
    client: httpx2.AsyncClient, session: AsyncSession
) -> None:
    story = await a_story(session)
    await session.commit()

    response = await client.delete(f"/stories/{story.id}")

    assert response.status_code == 204
    assert (await client.get(f"/stories/{story.id}")).status_code == 404
    kept = await session.get(Story, story.id, populate_existing=True)
    assert kept is not None and kept.deleted_at is not None
```

The test arranges through the session, acts through HTTP, and checks both the response and the row — all inside one transaction that leaves nothing behind.

> **THE .NET LENS**

`scope="session"` fixtures are `IClassFixture`/collection fixtures; the per-test rollback is what Respawn or a transaction scope gives you in .NET; the in-process client is `WebApplicationFactory.CreateClient()`. `async def` tests need a plugin, `pytest-asyncio`, configured here to run everything on one event loop — the engine's connections belong to it.

Tests that need a character use the **dummy**: "The Gilded Heron", a fictional card of about 1,850 words with an opening and a persona, under `tests/fixtures/dummy/`. airp's tests used a four-word card inline and missed a bug that cost a real story twenty-four turns of memory; from step 5 on, the memory tests here run on a card the size of a real one.

In CI the same Postgres image runs as a *service container* beside the job, on the same port, so the tests' default connection string works there unchanged.

TRY IT

Bring it all up and use it:

```
cp .env.example .env                    # fill in a username, a password, a token secret
docker compose up -d
uv run alembic upgrade head
uv run python scripts/seed_dummy.py     # prints the dummy character's id
uv run uvicorn lustjinn.main:app --reload
```

Open `http://127.0.0.1:8000/docs`. Call `POST /auth/sign-in`, copy the `access_token`, press **Authorize** and paste it. Then `POST /stories` with a name and the character id the seed script printed — the response's first message is the opening.

Now try to break the rule. In another terminal:

```
docker compose exec db psql -U lustjinn -c "UPDATE messages SET text = 'rewritten'"
docker compose exec db psql -U lustjinn -c "DELETE FROM messages"
```

Both are refused, by the database, with no application running in between.

2.10 What step 2 leaves behind

- Typed settings that stop the app at start-up when something is missing.
- A local Postgres, four tables, two migrations, and a test that keeps models and migrations in step.
- Messages that cannot be rewritten or deleted, whoever asks.
- A stories API behind a bearer token, with every route's guard tested from the app's own schema.
- A test suite that runs against real Postgres and leaves no trace.

Next — Chapter 3, One turn against OpenRouter. The first conversation with a model: an async HTTP client, the reply streamed to the caller as it is written, the reader's turn saved before the model is asked, and a ledger row for what it cost.

CHAPTER 3

One turn against OpenRouter

Step 2 left a story that could be opened and could not be played. Step 3 plays it: the reader sends a message, the API asks a model for the reply and streams it back word by word, and both sides of the exchange are kept — the reader's turn *before* the model is asked, the reply with the model that wrote it, the host that served it, and what it cost. Around that core, three pieces of turn mechanics: a retry that cannot store a message twice, a reroll that hides the old reply and keeps it, and slash commands, where a typo is refused instead of billed.

This chapter is about **async I/O**: an HTTP client that streams, a response that streams, and the generators that connect the two. It is also where money enters the code, and where a provider's failure modes get names.

3.1 One endpoint, one key

Every model call goes to **OpenRouter**, which fronts many models behind one OpenAI-compatible endpoint and one key. Choosing a model is configuration; so is pointing at a different provider that speaks the same shape. The settings grow accordingly (`src/lustjinn/settings.py`):

```
openrouter_api_key: SecretStr | None = None
openrouter_base_url: str = "https://openrouter.ai/api/v1"
model: str = "deepseek/deepseek-v4-flash"
temperature: float = Field(default=1.0, ge=0, le=2)
max_tokens: int = Field(default=1024, ge=1, le=32768)
model_timeout_seconds: int = Field(default=180, ge=1, le=3600)
prefer_providers: list[str] = []
ignore_providers: list[str] = []
allow_provider_fallbacks: bool | None = None
think_before_replying: bool = False
```

The key is **optional** here on purpose. The app must start without one — the stories API has nothing to do with the model — and a model call without a key fails before anything is sent, with an error naming the variable. `temperature` and `max_tokens` carry ranges, like `token_days` did. `think_before_replying` is off, which sends OpenRouter's `reasoning: {enabled: false}` on every reply: without it, the default model came back empty three times in five, every token spent thinking.

The two provider lists are the one setting that makes a measurable difference to the bill. OpenRouter spreads a model across hosts, and they are not interchangeable: some cache prompts, some do not, some return garbage. The lists go out as OpenRouter's `provider` field, trimmed and otherwise exactly as typed — the router drops a slug it does not know without complaint, so lower-casing to be helpful would only hide the typo.

3.2 Streaming, in both directions

A reply takes seconds to write. Showing it as it is written is the difference between an app and a form, so the model is asked to stream — and the API streams to its own clients in turn. Both use the same format, **server-sent events**: one long HTTP response of `text/event-stream`, carrying events a few lines at a time, each ended by a blank line.

```
event: delta
data: {"text": "She looks up"}

: a line starting with a colon is a comment — OpenRouter sends these as keep-alives

data: [DONE]
```

`src/lustjinn/sse.py` reads it and writes it, in about forty lines:

```python
async def events(lines: AsyncIterable[str]) -> AsyncIterator[Event]:
    name: str | None = None
    data: list[str] = []
    async for raw in lines:
        line = raw.rstrip("\r\n")
        if not line:
            if data:
                yield Event("\n".join(data), name)
            name, data = None, []
        elif line.startswith(":"):
            continue
        else:
            field, _, value = line.partition(":")
            value = value.removeprefix(" ")
            if field == "data":
                data.append(value)
            elif field == "event":
                name = value
    if data:
        yield Event("\n".join(data), name)
```

This is the first **async generator**: an `async def` with `yield` in it. Calling it runs nothing; it returns an object that produces one `Event` each time the caller asks with `async for`, and between two events the function is suspended exactly where its `yield` is, holding its local variables. The `lines` it reads are themselves produced the same way, by the HTTP response, as bytes arrive on the socket.

> **THE .NET LENS**
>
> An async generator is a method returning `IAsyncEnumerable<T>` with `yield return` inside, and `async for` is `await foreach`. The mental model carries over whole: lazy, pull-based, suspended between items. One difference — there is no `IAsyncDisposable` on the consumer's side to think about; leaving the `async for` early is enough, and `aclose()` on the generator is called for you when it is garbage-collected or the `async with` it lives in ends.

> **WATCH OUT**
>
> The orientation chapter's map said **respx** fakes HTTP in tests. It does not any more: respx supports `httpx`, and this project moved to **httpx2** in step 1, when Starlette asked for it. The map

> is corrected in this edition. The tests fake the network with httpx2's own `MockTransport` instead, which turns out to be simpler — a function from request to response, the exact shape of a fake `HttpMessageHandler`.

3.3 The client

`src/lustjinn/openrouter.py` holds one class with three methods: `stream`, which does the work; `complete`, which gathers a stream into one reply for callers with nothing to show while it is written; and `models`, which lists what OpenRouter serves. The heart of it:

```
async def stream(self, messages, *, model=None, temperature=None, max_tokens=None,
                 frequency_penalty=None, reasoning=None) -> AsyncIterator[str | Reply]:
    body: dict[str, object] = {
        "model": model or settings.model,
        "stream": True,
        "temperature": ...,
        "max_tokens": ...,
        "messages": [{"role": m.role, "content": m.content} for m in messages],
    }
    if frequency_penalty is not None:
        body["frequency_penalty"] = frequency_penalty
    if reasoning is not None:
        body["reasoning"] = {"enabled": reasoning}
    if (routing := self._routing()) is not None:
        body["provider"] = routing

    text: list[str] = []
    async with self._request("POST", "chat/completions", body) as response:
        async for event in sse.events(response.aiter_lines()):
            if event.data == "[DONE]":
                break
            chunk = _parse(event.data)
            ...
            for choice in _objects(chunk, "choices"):
                delta = _object(choice, "delta") or {}
                if piece := _string(delta, "content"):
                    text.append(piece)
                    yield piece
            usage = _object(chunk, "usage") or usage

    yield Reply(text="".join(text), model=..., provider=..., cost=_decimal(usage, "cost"), ...)
```

Things to notice, in the order they appear.

Keyword-only parameters. The `*` in the signature means everything after it must be passed by name: `stream(messages, temperature=0.4)`, never `stream(messages, 0.4)`. The donor project had three bugs from a positional argument added in the middle of a long list; here that mistake cannot be typed.

Fields sent only when asked for. `frequency_penalty` absent and `frequency_penalty: 0` are different requests to some backends; only absence means "no opinion". `reasoning` and `provider` are OpenRouter's own fields, so a backend that refuses unknown keys never sees them unless something is configured. The walrus `:=` assigns and tests in one expression, the way `if (Routing() is { } routing)` does in C#.

The stream yields two kinds of thing. Pieces of text as they arrive, then — once — a `Reply` with the whole text and the accounting. The return type says so: `AsyncIterator[str | Reply]`. The caller tells them apart with `isinstance`, which pyright uses to narrow the type on each branch.

```
async for piece in openrouter.stream(messages, ...):
    if isinstance(piece, Reply):
        written = piece
    else:
        yield format_event("delta", {"text": piece})
```

> **THE .NET LENS**
>
> `str | Reply` is a union type: a value that is one or the other, with the checker tracking which. C# has no unions; the nearest is a `OneOf<string, Reply>` or a base record with two subtypes and a type pattern in a `switch`. `isinstance` narrowing is the `is` pattern: after `if (piece is Reply written)`, `written` is a `Reply` — pyright does the same without the second name.

`Reply` is a frozen dataclass. `@dataclass(frozen=True, kw_only=True)` generates the constructor, equality and `repr` from the annotated fields, refuses assignment after construction, and requires every field by name. It is a `record` with `init`-only properties. Its `cost` field is a `Decimal`.

3.3.1 Money is a `Decimal`

JSON has one number type, and Python reads it as `float` by default: `0.0002` arrives as the nearest binary fraction, and a few hundred of those summed give `0.0006000000000000001`. A report about money that has to apologise for its own arithmetic is not one anybody checks against an invoice. So the parser reads every number with a decimal point as `Decimal`:

```
parsed: object = json.loads(text, parse_float=Decimal)
```

and the cost is converted nowhere else. In Postgres it is a `NUMERIC(18, 10)`, and SQLAlchemy maps that to `Decimal` on the way back.

> **THE .NET LENS**
>
> `decimal.Decimal` is `System.Decimal`, with one difference that matters here: it is arbitrary precision rather than 96 bits, and its precision is a context setting rather than fixed. Nothing in this project is affected; everything is ten decimal places or fewer.

3.3.2 Reading `Any` without lying to the checker

`json.loads` returns `Any` — a value the checker gives up on. In strict mode every use of it is an error, which is the point: a key that is not there or a value of the wrong type should be a decision, not a crash in a log. Four small functions make the decisions, once:

```
Json = dict[str, object]

def _object(source: Json, key: str) -> Json | None:
    value = source.get(key)
    return value if isinstance(value, dict) else None
```

```python
def _string(source: Json, key: str) -> str | None:
    value = source.get(key)
    return value if isinstance(value, str) else None
```

Everything the client reads goes through them, so a response shaped differently than expected produces `None` fields, never an exception — and the one place that must not be `None`, the text, is checked explicitly.

> **THE .NET LENS**
>
> `JsonNode` with `?["key"]?.GetValue<string>()` does the same job in the donor, null-conditional all the way down. The Python version is more explicit because `Any` is the one type the checker refuses to reason about: it has to be narrowed by hand, with `isinstance`, into something it can.

3.3.3 Every failure has one shape

The model can fail in six ways, and the client turns each into one `ModelError` whose message is fit to show the reader:

What happened	What the error says
No key configured	Names the variable. Fails before any request
Timeout, connection refused	"did not answer within 180 s", "could not reach …"
A 4xx or 5xx	The status, and `error.message` from the body — never the body, which echoes the prompt
A 200 that is not a stream	"not a stream (text/html)": a gateway's error page
An error chunk mid-stream	What the chunk said. A host dying mid-reply looks like this
A 200 with no text	`finish_reason`, the host, and "reasoning only" when the model spent everything thinking

The last one is the subtle one. A reply that is an empty string would be stored as though the model had answered with silence — a turn that never happened, permanent and billed. So no text is a failure, and the message says which of three different problems it was: a content filter (`finish_reason: content_filter`), a ceiling hit before the first token (`length`), or a host with nothing to say (`stop`).

One timeout, in one place. The donor had two — its own and `HttpClient`'s 100-second default — and the wrong one fired first. Here `httpx2.Timeout(settings.model_timeout_seconds)` is set on the request and nowhere else. For a stream it bounds the connection and the gap between one chunk and the next: a reply that keeps arriving is alive however long it takes as a whole.

3.3.4 An `async with` of our own

The request is opened in a class with `__aenter__` and `__aexit__`: enter sends the request, checks the status and hands back the open response; exit closes it, whether the stream was read to the end

or abandoned at an exception. `async with self._request(...) as response:` is the whole of the caller's responsibility.

> **THE .NET LENS**
>
> `__aenter__` / `__aexit__` are `IAsyncDisposable` plus a factory, used with `await using`. Python splits "acquire" and "release" into two methods on the same object, and the `with` statement guarantees the second runs — the same `try/finally` a `using` expands to.

3.4 The ledger

Four kinds of call spend money: a reply, a question asked out of character, and (from step 6) a summary and a fact extraction. Two of those fire without the reader asking for anything, and were invisible in the donor for as long as spending was inferred from message rows. So every billed call writes one row to `spend`, in `src/lustjinn/ledger.py`, whatever becomes of what it produced:

```
def row(story_id, kind: SpendKind, reply: Reply, message_id=None) -> Spend:
    return Spend(story_id=story_id, kind=kind, message_id=message_id,
                 model=reply.model, provider=reply.provider, generation_id=reply.generation_id,
                 prompt_tokens=reply.prompt_tokens, completion_tokens=reply.completion_tokens,
                 cached_tokens=reply.cached_tokens, cache_write_tokens=reply.cache_write_tokens,
                 cost=reply.cost)
```

Three decisions are in the table itself:

- **`cost` is nullable.** Null is "the API did not say what it charged"; zero is a price. A report that added them together would be confidently wrong instead of honestly incomplete.
- **No foreign keys.** Not to the story, not to the message. Erasing a story for good (step 7) takes its messages and keeps its ledger: what the router charged exists nowhere else once the response is gone.
- **Insert-only**, by a trigger like the one on messages — but with no purge flag. Migration `0003` refuses `UPDATE`, `DELETE` and `TRUNCATE` outright.

A reply's row points at its message, which is how a later cost report will know the reply was rerolled away: by reading the message's `deleted_at` at report time, never by rewriting the row.

> **WORTH KNOWING**
>
> The ledger is the one table that is not derived from anything. Everything else — summaries, facts, embeddings — could be rebuilt from the transcript. The bill could not.

3.5 The turn

`POST /stories/{id}/send` with `{"text": "I come in from the rain."}`. What comes back is the stream: `delta` events while the reply is written, then one last event that says how it ended.

```
event: delta
data: {"text": "She looks up from the ledger"}

event: delta
data: {"text": ", and does not smile."}
```

```
event: done
data: {"kind": "turn", "sent": {...}, "reply": {...}, "replayed": false}
```

The last event is `done` with both messages as stored, or `error` with a reason and — this is the important part — the reader's message, which was kept. Once a response has started streaming its status code cannot change, so every failure after that point is an event in the stream, not an HTTP status. A 4xx before the stream (a blank message, an unknown command, a story that does not exist) is still a plain JSON error.

3.5.1 Persist first

The order inside `turn()` in `src/lustjinn/turns.py` is the lesson the donor paid for:

1. Compute the request's hash and look it up (next section).
2. If the message is new, **insert it and commit** — before the model is called.
3. Build the prompt, call the model, stream the deltas.
4. On a reply: insert the assistant's message and its ledger row, **one commit**.
5. On a failure: nothing to undo. The event says *"Your message was kept … Do not send it again."*

If the send were only stored after a successful reply, a provider outage would silently swallow what the reader typed — and telling them it failed outright is how the same message gets typed a second time. A test pins each half: `test_the_message_survives_a_model_that_fails` and `test_retrying_a_send_the_model_failed_does_not_store_it_twice`.

3.5.2 Idempotency, anchored on the last reply

A retry must find the message it is a retry of. The message carries a **request hash**:

```python
def request_hash(story_id, anchor, text, instruction=None) -> str:
    directed = text if not instruction else f"{text}\x1f{instruction.strip()}"
    digest = hashlib.sha256(f"{story_id}|{anchor}|{directed}".encode()).hexdigest()
    return digest[:32].upper()
```

The **anchor** is the sequence number of the last visible reply — the state the message answers — and not the next free position. After a send fails at the model the reader's turn is already stored, so the next free position has moved; anchoring on it would give the retry a different hash and store the sentence twice. Anchored on the reply, the retry hashes the same, finds the unanswered row, and asks again against it. And the same words typed again *after* a reply has landed anchor differently: saying "Hello." twice in one story is ordinary, not a retry.

Four cases, in the order the code checks them:

The lookup finds…	What happens
Nothing	Insert the message, commit, call the model
A hidden message	Clear its hash, treat as nothing: re-sending words the reader deleted is how a turn is taken back and asked for again
A message with a visible reply after it	Return the stored pair, `replayed: true`, **no model call**
A message with no reply	Reuse the row and call again

The third case is reachable only by two copies of one request in flight, since the anchor moves once a reply lands; the test pins the hash with `monkeypatch.setattr` to simulate it. The second case came from a real story: the reader deleted their last turn and pasted it again, the hash matched the tombstone, and the model kept answering the story without that turn.

> **THE .NET LENS**
>
> `hashlib.sha256(...).hexdigest()` is `Convert.ToHexString(SHA256.HashData(...))`. The `\x1f` separator between text and instruction is a control character because a pipe or a space can appear in prose, and a separator the message can contain lets two different sends hash alike.

3.5.3 The response is a generator too

The endpoint returns a `StreamingResponse` wrapping an async generator; FastAPI iterates it and writes each string to the socket as it is produced. Two details:

```
# src/lustjinn/deps.py
StreamingSession = Annotated[AsyncSession, Depends(get_session, scope="request")]
```

`scope="request"` keeps the session dependency open until the response has been sent. The default scope closes a `yield` dependency when the endpoint *function* returns — which, for a streaming response, is before the stream has run. The generator would find its session closed. This is one of the few places where FastAPI's model of a request differs from the pipeline you know, and it is worth remembering whenever a response outlives its handler. It has its own name for that reason: a streamed route asks for `session: StreamingSession`, and one that finishes before it returns asks for `session: Session`.

The other is a header, `X-Accel-Buffering: no`, which tells a proxy in front (Render's, nginx) to pass each event on as it comes instead of collecting the body first.

3.5.4 The prompt, for now

Step 5 builds the context builder: layers in order, a token budget, the audit. Until then `src/lustjinn/prompt.py` sends the first two layers and the last, in the order it will keep: the character's card as a system message, the persona behind its frame ("The user is playing the following person…"), the visible transcript as `user` and `assistant` turns, and, when there is one, an instruction for this call only. The instruction goes as `user` when the transcript ends on a reply and as `system` when it ends on the reader's own turn — two user turns in a row, and a model tends to answer the second and forget the first.

Every story plays on the configured model. Step 7 let a story name its own, with the temperature mapped onto that model's range; chapter 7 says why it was taken out again.

3.6 Reroll

`POST /stories/{id}/reroll` writes the newest reply again. The old one is **hidden before the call** — a model shown its last attempt writes it again — and if nothing arrives to replace it, shown again, so a failed reroll never leaves the story shorter than it found it. Hidden, not removed: the row stays, and so does its ledger row. Both calls were charged.

Two refusals, both 409: there is no reply to reroll (the newest visible message is the reader's, or there is none), and the reply is the **opening**. The opening is a page a person wrote, and it sits at the top of the story for the rest of its life; a reroll there would trade it for a guess. It is told apart by two things, because either alone is wrong: no model wrote it (`model` is null — the reason step 2 stored that null), and the reader has not taken a turn yet. A beat the model writes after the opening with no reader turn (carry on, step 7) satisfies the first and not the second, and is rerolled.

3.7 Slash commands, and `/ask`

A message is permanent and billed. `(OOC: skip to the evening)` typed into one reaches the model, is stored forever, is counted in every later prompt, and may be summarised as something that happened. A command routes the same words where they belong — or nowhere — and leaves the transcript alone. The rule that follows: **an unrecognised command is refused, never sent.** A typo would otherwise cost what the message would have cost and land in the story as nonsense the character has to react to, and append-only means it could not be taken back.

`src/lustjinn/commands.py` reads what was typed and says what it is:

```
@dataclass(frozen=True)
class Prose:
    text: str

@dataclass(frozen=True)
class Command:
    spec: Spec
    argument: str

@dataclass(frozen=True)
class Unknown:
    name: str

@dataclass(frozen=True)
class Incomplete:
    spec: Spec

Parsed = Prose | Command | Unknown | Incomplete
```

Four small records and a union of them: the parse result is one of four things, and the caller has to say what it does with each. The endpoint does so with a `match` statement:

```
match commands.parse(body.text):
    case commands.Prose(text):
        return streams.streamed(turn(session, openrouter, settings, story, text))
    case commands.Command(spec=commands.Spec(name="ask"), argument=question):
        return streams.streamed(asides.ask(session, openrouter, settings, story, question))
    case commands.Incomplete(spec=spec):
        raise HTTPException(422, f"Usage: {spec.usage} — nothing was stored.")
    case commands.Unknown(name=name):
        raise HTTPException(422, f"/{name} is not a command, so nothing was sent ...")
```

> **THE .NET LENS**
>
> This is a `switch` expression with type patterns and property patterns: `case Command { Spec.Name: "ask" } c`. Python's version matches on the class, then on

> positional fields (`Prose(text)` binds the first field) or named ones (`spec=...`). Dataclasses get the positional form for free. The union `Parsed` is the closed set of cases a discriminated union would give you, minus the compiler's exhaustiveness check — pyright does not insist every case is handled, so the catch-all is the author's job.

The parser itself follows rules the donor arrived at one bug at a time: only a slash in the very first column counts (a slash mid-sentence is punctuation — `and/or`, a date — and prose is far more common than commands); `//` sends a line that genuinely begins with one, with the first slash stripped; the name runs to the first whitespace, a newline included, so a command whose argument starts on the next line is still that command; a command that needs an argument and has none is refused quoting its usage. The table of commands holds one entry for now, `ask`; each later step adds its own as it builds them, and until then `/do` is unknown here, and refused.

3.7.1 A question is not a turn

`/ask Where does she live?` asks the model a question about the story, out of character. The rules, each pinned by a test in `tests/test_asides.py`:

- **The same prompt as a turn**, byte for byte, up to the instruction — then the ask directive, which says what it is ("Step out of the scene. This is a question from the reader… it is not a turn"), what the answer must be (brief, plain, in the author's voice), and that it must come only from what was given: "Where it does not say, say that it does not say." Every instruction this project sends says what it is; a bare directive gets echoed back as the reply.
- **Cold and short**: temperature 0.4, 600 tokens, reasoning off, no frequency penalty. An answer can be promoted to a pinned fact in step 7, so an embellishment would become something the character believes.
- **Stored apart.** The answer streams back like a reply and goes into a new table, `asides` (migration `0004`), with the model, the host, the token counts and the newest message's sequence at the time. Nothing goes into `messages`, so no later prompt — and later, no summary, no fact, no embedding — ever sees it. A ledger row of kind `aside` is written, because a billed call that left no trace would make the audit quietly stop adding up.

On a caching host the question is nearly free: the prefix is what the next turn is about to send anyway.

3.8 Testing a stream

The model is faked at the network, not above it. `tests/scripted_model.py` is a queue of answers in OpenRouter's own stream format — a keep-alive comment, the text split mid-sentence across two chunks (a client that only handled whole words would pass with one chunk and garble a real reply), a chunk with `finish_reason`, a last chunk with `usage` and `cost`, then `[DONE]` — behind an `httpx2.MockTransport`:

```
def transport(self) -> httpx2.MockTransport:
    def handle(request: httpx2.Request) -> httpx2.Response:
        self.requests.append(request)
        self.calls.append(json.loads(request.content))
        return self._answers.popleft()()
    return httpx2.MockTransport(handle)
```

A test says what the model answers before the call that needs it, then asserts on what was sent:

```
model.fails().says("I could say the same about you.")

first = await send(client, story.id, "Hello.")
second = await send(client, story.id, "Hello.")

assert first.last[0] == "error"
assert second.done["reply"]["text"] == "I could say the same about you."
assert len(model.calls) == 2
```

`model` is a fixture, and the `client` fixture overrides `get_openrouter` with a client built on the scripted transport — the same `dependency_overrides` trick as the session. The vocabulary (`says`, `says_unpriced`, `fails`, `empty`, `truncated`) is the donor's, because its tests are the specification: every idempotency case in `tests/test_turns.py` has a namesake in `custom-airp`.

`tests/streaming.py` reads the API's own stream the way a client will — through the same `sse.events` parser the client uses to read OpenRouter — and answers the three questions a test asks: what was streamed (`.text`), how it ended (`.done`, `.error`), and with what status.

THE .NET LENS

`MockTransport` is a fake `HttpMessageHandler`: a function from `HttpRequestMessage` to `HttpResponseMessage`, injected under the client. The donor's `ScriptedHandler` is the same thing in thirty lines of C#; here it is a closure.

`monkeypatch.setattr(turns, "request_hash", pinned)` — used once, to simulate a true replay — is the one place a test reaches into a module and replaces a function, which NSubstitute would need an interface for.

TRY IT

Play a turn and watch it arrive. Put your OpenRouter key in `.env` as `LUSTJINN_OPENROUTER_API_KEY`, run the migrations and the seed script again, start the server, and create a story with the dummy character on `/docs` as in chapter 2. Then, with the token and the story's id, in a terminal (`-N` keeps curl from buffering):

```
curl -N -X POST http://127.0.0.1:8000/stories/<id>/send \
  -H "Authorization: Bearer <token>" -H "Content-Type: application/json" \
  -d '{"text": "I come in from the rain and shake out my coat."}'
```

The `delta` events appear as the model writes; the `done` event ends them. Open the story on `/docs`: both messages are there. Then:

- Send `/ask What does she make of me so far?` — an answer, and nothing new in the story.
- Send `/aks anything` — refused, with nothing stored and nothing billed.
- Reroll with `POST /stories/<id>/reroll`, then `SELECT sequence, role, deleted_at FROM messages` in psql: the old reply is hidden, its number is not reused, and `SELECT kind, cost FROM spend` shows every call that was made.

Finally, change the key in `.env` to something wrong, restart, and send again: the `error` event says the message was kept, and `GET /stories/<id>` agrees.

3.9 What step 3 leaves behind

- A streaming OpenRouter client with one error type, one timeout, provider routing, and money read as `Decimal` from the response — never computed from a price list.
- A ledger with a row for every billed call, insert-only, surviving everything.
- A turn that keeps the reader's message before the model is asked, is safe to retry, and streams the reply to the client over server-sent events.
- Reroll, and slash commands where a typo costs nothing. `/ask` answers out of character and stores nothing in the story.
- A scripted model that speaks OpenRouter's stream format, so every test runs without a network.

Next — Chapter 4, The library. Characters with their openings, personas and snippets, with optimistic concurrency and a history of every saved version — and the first foreign keys the owner edits through.

CHAPTER 4

The library

Step 3 played a story with a character that a script had seeded. Step 4 gives the owner the editor: characters with their openings, personas and snippets, on three shelves behind one set of endpoints, with the rules that make editing safe — a save that cannot quietly undo another one, a history that keeps every version, and a delete that the database itself refuses while a story still needs the entry.

This is the chapter on **constraints**: what the schema enforces on its own, what the ORM enforces on every write, and the little that is left for code to check. Along the way, the first mixin, a generic dataclass, and what `UPDATE … WHERE version = …` buys.

4.1 Three shelves, one shape

A character, a persona and a snippet are the same shape of thing: a name, a page of text, a version, a time of last change. A character also has an opening. In the donor they were four folders of text files; here they are three tables that share four columns through a **mixin** (`src/lustjinn/models.py`):

```
class LibraryEntry:
    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=new_id)
    name: Mapped[str] = mapped_column(String(200))
    version: Mapped[int] = mapped_column(server_default="1")
    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), onupdate=func.now()
    )


class Character(LibraryEntry, Base):
    __tablename__ = "characters"
    __table_args__ = (Index("uq_characters_name_lower", text("lower(name)"), unique=True),)

    card: Mapped[str] = mapped_column(Text)
    opening: Mapped[str | None] = mapped_column(Text)


class Snippet(LibraryEntry, Base):
    __tablename__ = "snippets"
    __table_args__ = (Index("uq_snippets_name_lower", text("lower(name)"), unique=True),)

    text: Mapped[str] = mapped_column(Text)
```

`LibraryEntry` is not a table. It is a plain class whose annotated attributes SQLAlchemy copies into each class that inherits it — every table gets its own `id`, `name`, `version` and `updated_at` columns. The unique index on `lower(name)` stays on each class, because it names the table.

> **THE .NET LENS**
>
> A mixin is the abstract base entity that EF Core maps with TPC (table per concrete type): each derived class gets a full table with the base's columns, and the base itself has none. The difference is that nothing here is polymorphic — there is no query over "all entries" — which is why it is a mixin rather than a mapped base. Multiple inheritance, `class Character( LibraryEntry, Base)`, is ordinary Python; the mixin goes first so its attributes win.

The endpoints are written once, for a **shelf**, and the shelf is a value (`src/lustjinn/library.py`):

```python
@dataclass(frozen=True)
class Shelf[E: LibraryEntry]:
    slug: Slug
    kind: str
    model: type[E]
    text_of: Callable[[E], str]
    set_text: Callable[[E, str], None]


SHELVES: dict[str, Shelf[Any]] = {
    "characters": Shelf("characters", "character", Character, _card, _set_card),
    "personas": Shelf("personas", "persona", Persona, _text, _set_text),
    "snippets": Shelf("snippets", "snippet", Snippet, _text, _set_text),
}
```

`Shelf[E: LibraryEntry]` is a **generic class** with a bounded type parameter, in the syntax Python 3.12 introduced: `E` is any subclass of `LibraryEntry`, and `model: type[E]` is the class itself, used as `select(shelf.model)` and `shelf.model(name=...)`. The two callables answer the one way the three tables differ — a character's text is its `card`, the others' is `text` — without a conditional at every use.

The path parameter picks the shelf, and its type is a `Literal` of the three slugs:

```python
Slug = Literal["characters", "personas", "snippets"]


def shelf_named(shelf: Slug) -> Shelf[Any]:
    return SHELVES[shelf]


OnShelf = Annotated[Shelf[Any], Depends(shelf_named)]


@router.get("/{shelf}/{entry_id}")
async def read_entry(shelf: OnShelf, entry_id: uuid.UUID, session: Session) -> EntryOut:
    ...
```

`GET /library/openings` answers 422 before any code runs: FastAPI validated the path against the `Literal`. And `shelf_named` is a dependency with a parameter of its own, which FastAPI reads from the path — a dependency that takes a path parameter is nothing special.

> **THE .NET LENS**
>
> `Shelf[E]` is `Shelf<E> where E : LibraryEntry`, and `type[E]` is `Type` with the generic argument kept — closer to a `Func<E>` factory plus a `typeof(E)` than to anything C# writes in

one token. The `Literal` on the path is a route constraint, `{shelf:regex(^characters|...$)}`, with the benefit that pyright knows the three values too.

4.2 Names

Four rules, each with a reason:

- **Unique without regard to case**, by the index on `lower(name)` from step 2. The endpoint checks first, to answer 409 with a sentence instead of a constraint name, but the index is what holds.
- **Refused:** `< > : " / \ | ? *` and control characters, on every shelf. A name may become a file name when a story is exported (step 7), and being refused a colon is a smaller surprise than an export that only works on one machine.
- **A snippet's name is one word** of letters, digits, `_`, `+` and `-`, up to 32 characters — because it is typed as `:name`, and anything else could not be.
- **A name starting with `_` is kept but not listed** unless asked for. A shelf holds working papers as well as things to play — a template, notes towards a character — and they belong beside what they are about rather than in every picker.

The last rule met a trap worth a box.

WATCH OUT

`select(Snippet).where(~Snippet.name.startswith("_"))` compiles to `name NOT LIKE '_%'`, and in SQL's `LIKE`, `_` matches *any one character*. The condition excluded every name. SQLAlchemy has the fix built in — `startswith("_", autoescape=True)` — but nothing warns when it is left out. A test that created two entries and listed one caught it; a list that happened to be tested with no hidden entries would not have.

The same rule had a second trap, and this one only showed on another machine.

WATCH OUT

The shelf is listed alphabetically. The first version said `ORDER BY lower(name)`, and its test passed where it was written. On the laptop's Postgres the same test failed: `_template` came out *between* `Elena` and `Zoe`.

`ORDER BY` on text does not have one meaning. Postgres sorts by the **collation** the database was created with, and collations disagree: the usual `en_US.utf8` ignores punctuation on a first pass, so a leading underscore vanishes; the `C` collation compares bytes, so `Ángela` lands after `Zoe`. The fix names the order instead of inheriting it:

```
def alphabetical(name: InstrumentedAttribute[str]) -> ColumnElement[str]:
    return name.collate("und-x-icu")
```

`und-x-icu` is ICU's language-neutral order — punctuation, digits, then letters without regard to case, accents beside their base letter — and it is the same on every server. A test now pins `_template, 9lives, alba, Ángela, Elena, Zoe` exactly.

THE .NET LENS

> SQL Server has the same hazard under another name: a column's collation (`SQL_Latin1_General_CP1_CI_AS` and friends) decides both ordering and equality, and a query that sorts one way on your machine sorts another on a server installed with a different default. `.collate(...)` is `EF.Functions.Collate(x, "...")`.

4.3 What a foreign key does for you

A story points at its character and its persona by id, `ON DELETE RESTRICT`. Two things follow, and neither needs code:

- **Renaming is free.** The donor stored the character's *name* in each conversation, so a rename broke every story using it. Here a `PATCH` with a new name reaches every story at once, and a new card reaches them from their next turn — `test_renaming_a_character_a_story_ uses_reaches_the_story` sends a turn after the rename and reads the new card in the prompt.
- **Deleting one a story uses is refused**, by Postgres, whoever asks.

The code's part is to say *which* stories, so the refusal is useful: `used_by` lists the live stories using an entry, and `DELETE` answers 409 with their names. It is asked at the moment of deleting, not trusted from when a screen was drawn — a story could have been started with it in between.

One case the key cannot express: a **deleted** story (`deleted_at` set) no longer counts as a user, but its row still points at the character, and the foreign key does not know the difference. So `used_by` says nobody, the delete goes to the database, the database refuses, and the endpoint turns that `IntegrityError` into its own 409: *a deleted story still refers to this character; it can go once that story is erased for good* — which is step 7's purge.

```
await session.delete(entry)
try:
    await session.commit()
except IntegrityError:
    await session.rollback()
    raise HTTPException(409, "A deleted story still refers to this ...") from None
```

> **THE .NET LENS**
>
> `IntegrityError` is `DbUpdateException` wrapping a `PostgresException` with SQLSTATE 23503. The pattern is the same: let the database be the last word, and translate its refusal where the caller can act on it. `raise ... from None` drops the inner exception from the traceback — the equivalent of throwing a new exception without an `InnerException`, on purpose, because the database's message is not the reader's business.

4.3.1 The default persona

A story that names no persona plays as the **default**, which the owner sets once. It lives in a new table, `settings`, with one row:

```
class AppSettings(Base):
    __tablename__ = "settings"
    __table_args__ = (CheckConstraint("id = 1", name="one_row"),)
```

```
    id: Mapped[int] = mapped_column(primary_key=True, default=1)
    default_persona_id: Mapped[uuid.UUID | None] = mapped_column(
        ForeignKey("personas.id", ondelete="SET NULL")
    )
```

A `CHECK (id = 1)` is how a table is made to hold one row at most. The foreign key is `SET NULL` rather than `RESTRICT`: deleting the default persona is allowed, and clears the setting — but only when nobody plays as it. The default counts as **used by every story that names none**, and `used_by` says so, so the delete is refused with those stories' names while any exist. That rule is the one piece of "who uses this" that the schema cannot carry, because the link runs through a setting rather than a column.

With no default set, a story that names no persona plays as nobody — even when exactly one persona is on the shelf. Choosing it would be choosing who the reader is playing, and it would change the day a second one appeared.

4.4 Optimistic concurrency

The owner edits a card on the laptop, then an hour later on the phone, from a page opened before the laptop's save. Saving the phone's text would silently undo the laptop's. The donor caught this by hashing the file; here the row has a **version**, and a save names the version it started from:

```
class EntryChanges(BaseModel):
    version: int
    name: Name | None = None
    text: Body | None = None
    opening: str | None = None
```

```
entry = await _one(session, shelf, entry_id)
if changes.version != entry.version:
    raise _changed_elsewhere(shelf, entry)
```

The refusal is a 409 that carries the entry **as it is now**, so a client can show both texts and let the reader decide — nothing is lost either way, which is the whole point. Saving again with the current version succeeds, and the overwritten text is in the history.

That check compares the client's version with the row as loaded. Between that load and the `UPDATE`, another save could still land. The ORM closes that gap by itself, with one mapper option on the mixin:

```
@declared_attr.directive
@classmethod
def __mapper_args__(cls) -> dict[str, Any]:
    return {"version_id_col": cls.version}
```

With `version_id_col` set, every `UPDATE` the ORM writes for these tables carries `WHERE version = <the version it loaded>` and sets `version = version + 1`. If no row matched, someone saved first, and the flush raises `StaleDataError` — which the endpoint answers with the same 409. `test_the_orm_checks_the_version_on_every_update_it_writes` bumps the version behind the session's back with raw SQL and watches the flush fail.

> **THE .NET LENS**
>
> `version_id_col` is `[ConcurrencyCheck]` on the property, or `IsConcurrencyToken()` in `OnModelCreating`, and `StaleDataError` is `DbUpdateConcurrencyException`. One difference: EF compares the *original* value it tracked; SQLAlchemy increments the column itself and compares the number. Either way the write carries the check, so there is no window.
>
> `@declared_attr.directive` marks a class-level setting a mixin computes per subclass — needed here because `cls.version` is a different column on each table. In EF the same thing would be a line in a base `OnModelCreating` helper called for each entity.

A save that changes nothing writes nothing: `session.is_modified(entry)` is false, no `UPDATE` is issued, the version stays, and no history row is added. The donor's test for this — `Saving_the_text_unchanged_does_not_replace_the_backup` — has its namesake in `tests/test_library_history.py`.

4.5 History

The donor kept one `.bak` of the last save. Here every version is kept, in `library_history`: kind, the entry's id, the version number, the name, text and opening as saved, and when. A row is written on create (version 1) and on every save that changed something, in the same transaction as the change:

```
if session.is_modified(entry):
    session.add(_remembered(shelf, entry, version=entry.version + 1))
    await session.commit()
```

Two decisions carry over from the ledger. **No foreign key**: the history of a deleted entry is still history, and `GET /library/personas/{id}/history` answers for an id that is no longer on the shelf. **Insert-only**, by a trigger (migration `0006`) that refuses `UPDATE`, `DELETE` and `TRUNCATE`. The history is the one thing in the library that cannot be edited.

> **WORTH KNOWING**
>
> Three tables now share the insert-only trigger pattern: `messages` (with a purge door), `spend` and `library_history` (without). It is the same twelve lines of PL/pgSQL each time, and that is fine — a trigger is schema, and schema is written out. The course mentions it so the repetition does not look like an oversight.

4.6 A note for lists, and a count for the tabs

Two small things were added to the library after step 9, both because the owner's library grew past what a list of names can carry: **a short description** on an entry, and **a count** on each shelf. They are in `src/lustjinn/library.py` and migration `0017`.

4.6.1 The description

The mixin from the first section is where it goes, so one declaration reaches four tables — characters, personas, snippets and the persona pieces of chapter 10:

```python
class LibraryEntry:
    ...
    description: Mapped[str | None] = mapped_column(String(200))
    """A short note for the owner, shown in lists. Never part of a prompt: nothing that builds one
    reads it."""
```

`library_history` is not a `LibraryEntry`, so its copy is a column of its own, and the migration adds five columns to five tables with one loop. The history row carries the description because a version is the whole entry as saved.

The rules sit in one function, shared by the library's two request bodies and the pieces':

```python
def check_description(description: str | None) -> str | None:
    if description is None:
        return None
    description = description.strip()
    if len(description) > DESCRIPTION_LENGTH:
        raise ValueError(f"A description is at most {DESCRIPTION_LENGTH} characters.")
    return description or None
```

Trimmed, blank is none, and over 200 characters is refused with a sentence. Pydantic runs it as a `field_validator`, so a refusal is a 422 whose body says what is wrong. The 200 is the same figure as the column's `String(200)`: the database would refuse a longer one too, but a person should be told in words.

The interesting rule is on the save. A save names only what it changes, so there are **three** states for the description: left out (do not touch it), `null` or blank (clear it), and a text (set it). JSON cannot tell "absent" from `null` once it is a plain field, and pydantic can:

```python
if "description" in changes.model_fields_set:
    entry.description = changes.description
```

`model_fields_set` is the set of fields the client actually sent. The opening of a character is saved the same way, and for the same reason. A save that changes only the description is a real save: `session.is_modified(entry)` is true, the version goes up and a history row is written, so the description has a history like everything else; a save that changes nothing still writes nothing.

The most important test is the one that is *not* about the description: it sends a turn with a character and a persona that both carry one and asserts the text is in none of the messages the model is sent. A note for the owner must not become an instruction to the model.

> **THE .NET LENS**
>
> `model_fields_set` is what `System.Text.Json` has no built-in answer for: a DTO with `string? Description` cannot say "the client left it out". In .NET you reach for a `JsonPatchDocument`, an `Optional<T>` wrapper, or a hand-rolled `JsonConverter` that records which properties it saw. Pydantic records it for free.

4.6.2 Counting the shelves

The web library's tabs show every shelf's count while one shelf is open, so a count per list call would take three calls. `GET /library/counts` answers all of them at once:

```
@router.get("/counts")
async def read_counts(session: Session) -> Counts:
    ...
    total = await session.scalar(select(func.count()).select_from(model))
    underscored = await session.scalar(
        select(func.count()).where(model.name.startswith("_", autoescape=True))
    )
```

A count is what a list shows, and a list hides names that start with an underscore (working papers, a template), so those are counted apart under `hidden` and left out of the number on the tab. `autoescape=True` matters for the same reason as in the list: `_` is a wildcard in SQL's `LIKE`, which `startswith` compiles to. The persona pieces are counted too; they have no hidden names.

One detail of FastAPI's routing: routes match **in the order they are declared**, and the shelves are `/{shelf}` with a `Literal` type. `/counts` is not a shelf, so if it were declared after `/{shelf}` it would be refused with a 422 before it was ever tried. It is declared first, and `test_counts_is_not_mistaken_for_a_shelf` pins both halves.

4.7 Snippets expand at send time

A snippet is authored prose deployed at the dramatically right moment: written once, typed as `:storm`, replaced by its text before the message is stored. `src/lustjinn/snippets.py` does the replacing, and the hard part is not finding a colon but knowing when a colon is *not* a trigger. Prose is full of them — `10:30`, `https://…`, `note: this` — and a message is permanent, so guessing is not allowed. The rules:

- The colon must **open a word**: preceded by whitespace or nothing. That excludes clock times and URLs.
- The name is letters, digits, `_`, `+`, `-`, up to 32 of them. A longer run is not a name cut short; it is not a name.
- A closed `:name:` is an **emoji shortcode**, which the clients handle, and the API passes through untouched. This was the open question on the issue; the decision is that emoji are presentation, not content, and belong where the typing happens.
- An unknown name is left exactly as typed. One pass: an expansion is not scanned again.

The expansion runs first in `send`, before the line is read for commands — so a trigger works inside a question too — and before the request hash, so a retry of `:storm` is still one turn. The stored text is the expanded text; a later edit of the snippet does not reach it, which is what "copied, never read again" means.

```
def expand(text: str, snippet: Callable[[str], str | None]) -> str:
    out: list[str] = []
    i = 0
    while i < len(text):
        opens_word = text[i] == ":" and (i == 0 or text[i - 1].isspace())
        if not opens_word:
            out.append(text[i])
            i += 1
            continue
        end = i + 1
        while end < len(text) and text[end] in NAME_CHARACTERS and end - i <= MAX_NAME_LENGTH:
            end += 1
```

```
        name = text[i + 1 : end]
        ...
```

`expand` takes the lookup as a function, `Callable[[str], str | None]`, rather than a table. The endpoint hands it a closure over the snippets it loaded; the tests hand it a dictionary through `by_name`. A function parameter is the cheapest seam there is.

> **THE .NET LENS**
>
> `Callable[[str], str | None]` is `Func<string, string?>`, and the scanner is a line-for-line port of the donor's `ShortcodeScanner.ExpandAll` with the emoji branch removed. The one Python idiom worth noticing is `text[i + 1 : end]` — slicing, half-open like `Range` in C#, and the spaces around the colon are what ruff's formatter does when the bounds are expressions.

4.8 What the tests look like now

The library tests are the first to use the API's own endpoints as the *arrange* step — a character created through `POST /library/characters` rather than inserted through the session — because the endpoints are now the natural way to make one. The two styles coexist: `a_character(session)` where the test is about something else, the endpoint where the test is about the library.

One test needed a line of care:

```
character_id = character.id  # the refused delete rolls the shared session back
```

The test and the app share one session, and the app's `rollback()` after the refused delete expires every object in it — including the test's own `character`. Reading `character.id` afterwards would try to reload it, which async code cannot do on its own. Capture what you need before the request; a test that shares a session with the code it tests shares its side effects too.

> **TRY IT**
>
> Rebuild the database and seed it, then edit the dummy from `/docs`:
>
> 1. `GET /library/characters` — one entry, "The Gilded Heron", `version: 1`.
> 2. `PATCH` it with `{"version": 1, "name": "The Heron"}`. The response says `version: 2`.
> 3. `PATCH` it again with `{"version": 1, "text": "x"}` — 409, and `current` is what it says now. Send `{"version": 2, "text": "x"}` instead: 200, `version: 3`.
> 4. `GET …/history` — three rows, newest first, with the card you just overwrote.
> 5. `DELETE` it: 409, naming the stories that use it. Delete the story first, then try again: 409 again, from the foreign key — a deleted story still points at it. That is the purge's job, in step 7.
> 6. `POST /library/snippets` with `{"name": "storm", "text": "Rain hammers the glass."}`, then send a turn whose text is `I come in. :storm` — the stored message carries the expansion. Send `at 10:30 :nothing` — stored as typed.

4.9 What step 4 leaves behind

- Three shelves behind one set of endpoints, with names unique without regard to case, hidden working papers, and the default persona that a story without one plays as.

- Renames that reach every story at once, and deletes the database refuses while a story needs the entry — with the stories named.
- Saves that carry the version they started from, checked twice: by the endpoint against the client's version, and by the ORM on the write itself.
- A history of every version, insert-only, that outlives the entry.
- Snippets that expand where `:name` opens a word, and nowhere else.
- Afterwards: a short description on every entry (never read by a prompt, versioned like the rest) and a count of each shelf in one call.

Next — Chapter 5, The context builder. The prompt, properly: layers in order from least to most volatile, a token budget counted with the model's own vocabulary, the newest turn always kept, and an audit of what each layer cost — mostly pure functions, and mostly tests.

CHAPTER 5

The context builder

Until now the prompt was three things in a row: the card, the persona, the whole transcript. That works for a story of twenty turns. It does not work for a story of two hundred, and the thesis of this project — *you do not need an infinite context window if you have a good retrieval layer* — needs a place where the prompt is *assembled* rather than concatenated. Step 5 builds that place: a module of pure functions that takes everything a prompt could hold and returns what the model is sent, within a budget, with an account of what it did.

Nothing in this step touches the database or the network. That is the point of it, and the lesson: the most important logic in the app is the easiest to test.

5.1 Why layers, and why this order

A model is sent a list of messages. The order of that list is a cache key: providers keep a **prefix cache**, and a request whose first *n* messages are byte-for-byte the same as the last request's gets those *n* messages for a fraction of the price. So the prompt is laid out from the thing that never changes to the thing that changes every turn:

Layer	Changes	Here since
character	when the card is edited	step 3
persona	when the persona is edited	step 3
directives	when a dial is turned	step 7
world	when a fact is extracted or retired	step 6
summaries	every few dozen turns	step 6
history	every turn — and it is the layer that *gives way*	step 3
memories	every turn	step 6
trackers	every turn	step 7
instruction	every turn; this turn only	step 3

Retrieval goes *after* the history on purpose. Recalled turns change with every message, and putting them before the transcript would invalidate the cache from that point down, every turn.

5.2 The shape: dataclasses in, dataclasses out

`src/lustjinn/context.py` has one input type and one output type:

```
@dataclass(frozen=True, kw_only=True)
class Layers:
    character: Character
    persona: Persona | None = None
    directives: str | None = None
    facts: Sequence[Fact] = ()
    summaries: Sequence[str] = ()
    history: Sequence[Message] = ()
    memories: Sequence[Recalled] = ()
    trackers: str | None = None
    instruction: str | None = None


@dataclass(frozen=True)
class Built:
    messages: list[ChatMessage]
    spent: dict[str, Spent]
    budget: int | None
```

A **dataclass** is a class whose `__init__`, `__repr__` and `__eq__` are written for you from the annotated fields. `frozen=True` makes instances immutable. `kw_only=True` means every field must be passed by name — `Layers(character=c, history=h)`, never `Layers(c, h)`.

The keyword-only rule is not taste. airp's `Build` took its layers positionally, and three of its bugs came from the same move: a new layer added in the middle of the parameter list, and one call site that kept passing the old arguments in the old positions — summaries where the directives should be, compiling cleanly. Names cannot be shifted.

> **THE .NET LENS**
>
> `@dataclass(frozen=True)` is a `record` with `init`-only properties; `kw_only=True` is a constructor you may only call with named arguments. `Sequence[Message]` is `IReadOnlyList<Message>` — a type that promises you will only read. `Fact` and `Recalled` are small records that steps 6 and 7 will produce; they exist now so the builder's shape does not change when they arrive.

5.3 Counting tokens

A budget is measured in tokens, and there is exactly one honest way to count them: the way the model does. `src/lustjinn/tokens.py`:

```
ENCODING = "o200k_base"
PER_MESSAGE = 4


@lru_cache
def encoding() -> tiktoken.Encoding:
    return tiktoken.get_encoding(ENCODING)


def count(text: str) -> int:
```

```
    return len(encoding().encode(text, disallowed_special=()))


def for_message(text: str) -> int:
    return count(text) + PER_MESSAGE
```

tiktoken is OpenAI's tokenizer library; `o200k_base` is the vocabulary of their current models, and the one airp counted with. Two details:

- `PER_MESSAGE = 4`: a message costs more than its text. The role and the separators the API wraps around it come to about four tokens — a figure measured against what a provider reported for a real transcript, not derived from anything.
- `disallowed_special=()`: by default tiktoken refuses to encode text containing a marker like `<|endoftext|>`, because in a prompt those mean something. In a card they are just characters. A character sheet that quoted one would have crashed the count.

> **WATCH OUT**
> A characters-per-token constant is not a count. Spanish runs about 30% more tokens per character than English in these vocabularies; a budget that holds in one language overflows in the other. That is why there is a tokenizer in the project and not a division by four.

> **WORTH KNOWING**
> tiktoken downloads the vocabulary file (3.6 MB) the first time it is used and keeps it in a cache directory — `TIKTOKEN_CACHE_DIR` if set, the system's temporary folder otherwise. On this machine and in CI that happens once and silently. A sandbox that blocks `openaipublic.blob.core.windows.net` cannot count tokens until the domain is allowed.

5.4 The budget

`build(layers, budget=32_000, recall_percent=10)` does its arithmetic in a fixed order:

1. **Fit the memories under their cap.** The cap is `budget × recall_percent / 100` — a *share*, not a count. Four recalled turns once filled a 60,000-token prompt, which is why the rule is not "at most four". Memories arrive most relevant first; each is kept if the layer still fits under the cap with it, and one that does not fit is **skipped, not fatal** — a shorter one further down may still make it. The survivors are then sorted back into story order, so the model reads them as a transcript.
2. **Cost every fixed layer.** Character, persona, directives, world, summaries, memories, trackers, instruction: these go in whole, whatever they cost. The instruction is counted here too — it is never squeezed out by old turns.
3. **Give the history what is left.** `room = max(0, budget − fixed)`. Walk the transcript from the newest message backwards, adding each turn while it fits, and **stop at the first that does not**. The kept turns are always a contiguous run ending on the newest.
4. **The newest message is always kept**, before the room is even checked. A prompt that leaves out the words the reader just typed is a reply to nothing — and a real story once lost the reader's own message that way. If the budget cannot hold even the fixed layers, the prompt goes *over* budget by one message rather than under it by the one that matters.

```python
def _fit_history(history: Sequence[Message], room: int | None) -> tuple[list[Message], int]:
    if not history:
        return [], 0
    kept = [history[-1]]
    used = tokens.for_message(history[-1].text)
    for turn in reversed(history[:-1]):
        cost = tokens.for_message(turn.text)
        if room is not None and used + cost > room:
            break
        kept.append(turn)
        used += cost
    kept.reverse()
    return kept, len(history) - len(kept)
```

The default budget is 32,000 tokens — far below any current model's window, on purpose. Two reasons, both measured on real stories: attention thins out as a prompt grows, so a longer transcript does not make a better reply past a point; and every token in the prompt is paid for on every turn, so a 100,000-token prompt costs three times what a 32,000-token one does, forever.

> **THE .NET LENS**
>
> `_fit_history` is the loop you would write with a `for` over `history.Reverse().Skip(1)` and a running total. Python's `reversed(history[:-1])` is the same `Reverse().Skip(1)` read inside-out: slice off the last element, then iterate backwards. The tuple return `(kept, dropped)` is a `(List<Message> Kept, int Dropped)` value tuple; the caller unpacks it as `history, dropped = …`.

5.5 The audit

Every `Built` carries `spent`: per layer, how many tokens it took and how many items were dropped to fit. `describe()` turns that into one line:

```
character 2755 · persona 412 · history 1830 (12 dropped) · instruction 60 · total 5057/32000
```

That line is stored on every reply, in `messages.context_audit`, with the total in `estimated_prompt_tokens` — right beside `prompt_tokens`, the figure the provider actually billed. `GET /stories/{id}/audit` lists the newest twelve replies and eight questions with both numbers. Rerolled replies are listed too, marked hidden: "why did it say that?" is asked of them more than of any other.

Having the estimate beside the bill is what made the next paragraph possible.

> **WATCH OUT**
>
> The first real turn of this project was billed at **2,755** prompt tokens. The counter said **3,163** for the same prompt — 15% high. Not a bug in the arithmetic: `o200k_base` is OpenAI's vocabulary, and DeepSeek has its own, which happens to spend fewer tokens on this prose. High is the safe direction (a prompt the counter thinks is at the budget is under it), so the test that checks the real turn pins the margin at "between the billed figure and 20% above it" rather than pretending to 10%. Closing the gap is #94, and the audit is collecting the data for it on every reply.

5.6 Testing pure logic

Thirty-five tests cover this step, and most run in under a millisecond, because the thing under test is a function from values to values. The tests build their inputs by hand:

```python
def turns(*lines: str) -> list[Message]:
    roles = [Role.USER, Role.ASSISTANT]
    return [Message(sequence=i + 1, role=roles[i % 2], text=line) for i, line in enumerate(lines)]


def test_the_turns_kept_are_the_ones_nearest_the_reply(dummy: Dummy) -> None:
    built = build(
        Layers(character=heron(dummy), history=long_story(100)),
        budget=tokens.for_message(dummy.card) + 500,
    )

    kept = [m.content for m in built.messages[1:]]
    assert kept[-1].startswith("Turn 100.")
    numbers = [int(line.split(".")[0].split()[1]) for line in kept]
    assert numbers == list(range(numbers[0], 101))
```

Three things to notice:

- `Message(...)` **without a database.** A SQLAlchemy model is still a Python object; constructing one costs nothing and saves nothing. The test never opens a session.
- **The card is the dummy's**, 1,850 words, through the `dummy` fixture. airp's equivalent tests used a four-token card inline, and a bug that only appeared with a real-size card cost a story twenty-four turns of memory. Here the fixed layers are big enough to make the budget bind the way it does in a real story.
- `*lines: str` is `params string[] lines`: `turns("One.", "Two.", "Three.")`.

> **THE .NET LENS**
>
> These are xUnit `[Fact]`s over a static class, and the fixture is `IClassFixture<Dummy>`. What has no counterpart is how little ceremony it takes: no `Arrange` helpers to inject, no `IOptions<ContextOptions>` to fake. `build` takes its budget as an argument, so a test passes a small number and the rule shows itself.

> **TRY IT**
>
> Play a turn, then look at what it was built from:
>
> ```
> GET /stories/{id}/audit
> ```
>
> Compare `estimated_prompt_tokens` with `prompt_tokens` on the newest reply. Then set `LUSTJINN_CONTEXT_BUDGET=4000` in `.env`, restart the server, play another turn, and look again: `history` now shows `(n dropped)`, and the total sits just under 4,000 — unless the card alone is bigger than that, in which case the total is over and the history is exactly one message. Put the budget back.
>
> From `psql`, the same comparison across every reply so far:

```
SELECT sequence, estimated_prompt_tokens, prompt_tokens,
       round(100.0 * estimated_prompt_tokens / prompt_tokens) AS estimate_pct
FROM messages WHERE prompt_tokens IS NOT NULL ORDER BY sequence;
```

5.7 What step 5 leaves behind

- One place where the prompt is assembled, in a fixed order, with slots for everything steps 6 and 7 will add.
- A token count in the model's own units, and a budget the transcript yields to — oldest first, newest never.
- Recalled memories capped at a share of the budget, ready for retrieval to fill.
- An audit on every reply, estimate beside bill, and the first calibration it produced.

Next — Chapter 6, The memory. The three mechanisms that let a story outlive its budget: summaries of what happened, retrieval of what was said, facts about what is true now — and the background calls that produce them only when a story needs them.

CHAPTER 6

The memory

This is the chapter the project exists for. Everything before it was a chat client with good habits; what this step adds is the reason a story can run to its five-hundredth turn and the character still knows what was said on the tenth.

> You do not need an infinite context window if you have a good retrieval layer.

The memory is three mechanisms, each answering one question, and a rule about when they run:

Mechanism	Answers	Produces	When
Summaries	what happened	prose, one per stretch of turns	when the transcript no longer fits the budget
Retrieval	what was said	the few old turns nearest to the reader's last words	once something has been summarised
Facts	what is true now	one sentence each, with a validity range	right after each summary

The rule: **a story that fits its budget costs no extra call.** Nothing here fires until the context builder from chapter 5 would otherwise start dropping turns.

6.1 One entry point

`memory.compose` is what a turn (and a question) now calls instead of `context.build`:

```
built = await memory.compose(
    session, openrouter, settings, story, persona, history, instruction=...
)
```

It does five things in order, and each is a section below:

1. Loads the story's summaries and works out which turns they already **cover**. Covered turns are not sent; the summaries stand in for them. The rest are the *recent* turns.
2. If the recent turns no longer fit, **compresses** the oldest of them into one new summary.
3. If that worked, **extracts facts** from the same stretch.
4. **Retrieves** the compressed turns nearest to the reader's last message.

5. Builds the prompt with chapter 5's builder — summaries, world, history, memories all in their layers — and returns it with the audit.

Steps 2, 3 and 4 are each a model call, and each can fail. The design decision that shapes the whole module is what happens then: **the reader never pays for a background failure with their reply.** A failed summary sends the turns whole and sets the budget aside; a failed extraction leaves the summary standing; a failed retrieval sends no memories. The reply is asked for regardless.

> **THE .NET LENS**
>
> `compose` is an orchestration method the way an application service in a clean-architecture .NET project is: it owns the order of operations and the transaction boundaries, and delegates the work. There is no mediator and no handler per step; the five steps are five function calls on one page, because being able to read the whole algorithm top to bottom is worth more here than pluggability.

6.2 Summaries: what happened

6.2.1 When, and how much

The builder decides what fits; the summariser has to decide *before* the builder runs, so it estimates the same thing: the fixed layers, plus the reply's ceiling, plus the room retrieval will take, plus 200 tokens of slack. What is left is the **allowance** for the recent turns.

```
def batch_to_compress(recent: Sequence[Message], allowance: int) -> list[Message]:
    overflow = overflowing(recent, allowance)
    if overflow <= 0:
        return []
    size = min(max(overflow, WORTH_A_CALL), len(recent) - ALWAYS_WHOLE, AT_MOST_PER_SUMMARY)
    return list(recent[:size]) if size > 0 else []
```

Three constants, each a lesson airp paid for:

- `WORTH_A_CALL = 10`. The first version compressed only what overflowed. On a long story that was two messages, every single turn — a model call per turn for a summary of two lines. A batch of at least ten buys several turns of room per call.
- `ALWAYS_WHOLE = 6`. The newest six turns are the scene in progress and stay verbatim, **whatever the overflow**. The first version of this step treated six as a limit on *widening* the batch only, as the donor does, and `max(overflow, …)` could reach past them. The real playtest caught it on the thirteenth turn of a story squeezed into a 6,000-token budget: the reader's own newest message was summarised, the prompt had no history layer at all, and the character answered a message it had never been shown. That is the exact failure the kickoff lesson forbids, so the newest six are now a hard cap (#96). What still does not fit among them is the context builder's problem — dropped from that prompt, the newest never — not the summariser's. In practice the case needs a budget that leaves almost no room for turns; at 32,000 the newest six never overflow.
- `AT_MOST_PER_SUMMARY = 40`. One call over ninety-nine messages came back as `##`. A model asked to compress too much compresses it to nothing.

6.2.2 The second chance: turns the builder had to drop

The summariser plans from an **estimate** made before the facts and the recalled turns are known: the fixed layers, the reply's ceiling, 200 tokens of slack and a guess at the room retrieval will take. When the real prompt comes out larger, the builder (chapter 5) drops the oldest turns that no summary covers, and for that one reply those turns are *nowhere*: not in a summary, not in the prompt, not in the facts (extraction runs on the same batch), and not recallable (only summarised turns are embedded). A real story showed it on 55% of its replies in one day: in one case a meeting, a phone number and a plan were missing for seven replies, until the next summary covered them. Part of the cause was that the world layer, 3,200 tokens of facts, was not in the estimate at all.

Two changes, both in `compose`:

- **The reserve now counts the world layer**, read before the summariser plans.
- **The prompt is built, and if it dropped turns, those are summarised and it is built again** (`dropped_to_compress`), up to `MOST_SUMMARIES_A_TURN = 3` summaries in all. The dropped turns were not in the prompt, so summarising them frees nothing, and the summary's own text would push the oldest turns out in turn; so the batch also takes enough of the turns after them to pay for a summary of the largest size. After that every turn is either in the prompt or inside a summary that is.

The failure rule is unchanged: if the second summary fails, the turns go whole and the budget is set aside. And the cost is a model call the reader waits for, which is why the loop has a ceiling and why this is the exception, not the plan.

> **THE .NET LENS**
> The shape is a retry loop with a postcondition: build, check the result for what must not happen (a silent drop), repair, build again. In .NET it reads like a Polly policy that retries on a result predicate, with the repair done in its retry callback.

6.2.3 What is asked, and what is accepted

The stretch is rendered as `Name: text` lines — the character by name, **the reader by their persona's name, never "User"** — and sent with an instruction that asks for a factual account: what happened in order, what was established, what changed between people, what was promised or left unresolved, where the scene was left. Temperature 0.3, because a creative summariser invents history the character then believes forever. A ceiling of 1,200 tokens.

The answer is judged before it is kept:

```
def credible(summary: str, source: Sequence[Message]) -> bool:
    produced = tokens.count(summary)
    source_tokens = sum(tokens.count(turn.text) for turn in source)
    return produced >= max(LEAST_SUMMARY_TOKENS, source_tokens // LEAST_COMPRESSION)
```

A summary shorter than one sixtieth of what it replaces cannot have kept it. Working summaries compress 3× to 19×; `##` compresses 600×. A refused summary is **still billed** — the call happened — and the turns go to the model whole this time. Over budget rather than discard.

> **WATCH OUT**

> Where a story is compressed matters as much as whether. The summary layer sits *before* the history in the prompt, so a model reads "earlier, this happened" and then the live turns — the same order a reader would. And one summary per call: a long backlog is worked through over several turns, not in one heroic request that comes back as `##`.

6.3 Facts: what is true now

A summary is prose, and prose is a poor place to keep "she is allergic to shellfish". So right after each summary, the same stretch is read once more by an **extractor**, which answers JSON:

```
{
  "facts": [ { "subject": "Isaure", "text": "owes Derrow & Sons four hundred and ten marks." } ],
  "retired": [ "01a111c5" ]
}
```

- **Subject** is always a name — a character, a place, a pair. The instruction forbids "User", "the user" and "the reader" as subjects: a story does not know who the user is.
- **New facts** begin at the first turn of the stretch; the world layer shows them from the very turn they were extracted on, because `compose` reads the live facts *after* the extraction.
- **Retiring** is the interesting half. The extractor is shown the existing facts with a short id each, and names the ones the stretch made false — a change of heart, a healed wound, a kept promise. A retired fact gets `valid_to` = the last turn of the stretch and leaves the prompt. A world state that only accumulated would end up asserting both that she distrusts the reader and that she trusts them, and the model would believe whichever it read last.
- **A pinned fact cannot be retired by the model.** When a person states a fact (step 7's `/fact`, or pinning an `/ask` answer), it has no `model` and `pinned` is true. It is not derived from the transcript — it may be about something the story never mentioned — so nothing in the transcript can prove it wrong.

The parsing is deliberately generous: the JSON is read from the first `{` to the last `}`, so a model that wraps its answer in a code fence or a sentence does no harm. Anything that is not the agreed shape is skipped rather than guessed at, and an answer with no object in it records nothing — billed, like the refused summary. Temperature 0.2, colder still: an invented fact is injected into every prompt from then on.

> **THE .NET LENS**
>
> `parse` is the kind of function `System.Text.Json` would make you write with a `JsonDocument` and a dozen `TryGetProperty` calls. Python's `json.loads` gives back nested `dict`s and `list`s, and `isinstance` checks do the rest. pyright strict insists on those checks: a value out of `json.loads` is `Any`, and the `# pyright: ignore` comments mark exactly where the untyped world is being read.

6.4 Retrieval: what was said

A summary keeps what happened; it does not keep *how it was said*, and three hundred turns later the character should still be able to echo the reader's own words from the tenth. That is retrieval: each

compressed turn is turned into a **vector** — a list of 1,536 numbers that places its meaning in a space where similar texts are near — and the reader's last message is used to find the nearest few.

6.4.1 pgvector

The vectors live in Postgres, in a column of type `vector(1536)` that the **pgvector** extension provides, with an HNSW index for approximate nearest-neighbour search by cosine distance.

```
distance = Embedding.vector.cosine_distance(wanted).label("distance")
rows = await session.execute(
    select(Message.sequence, Message.role, Message.text, distance)
    .join(Embedding, Embedding.message_id == Message.id)
    .where(Message.story_id == story.id, Message.sequence <= covered, Message.deleted_at.is_(None))
    .order_by(distance)
    .limit(settings.recall_count)
)
```

`cosine_distance` compiles to pgvector's `<=>` operator; `1 - distance` is the similarity, and anything below the threshold (0.35) is not recalled. The column refuses a vector of any other length — a test proves it with a three-number vector.

6.4.2 The rules

- **Only compressed turns are embedded.** The recent ones are in the prompt verbatim; recalling them would send them twice. Embedding is done lazily, oldest first, at most 128 per call — a long story meeting retrieval for the first time catches up over a few turns.
- **The query is the reader's last message** among the recent turns. What they just said is the best signal of what they are reaching back for.
- **Unless it is a pasted document** (more than `LONGEST_QUERY`, 500 tokens). One story had a reader who pasted a video post every few dozen turns: a thumbnail, a background essay, the lyrics, the staging. Every post resembles every other post as a whole, so *whatever part* of one was used as the query (the whole of it, the last 1,500 characters, the part the reader had never written before) the four nearest turns were the earlier posts and the crowd's reactions to them, five thousand tokens of other videos beside the one being answered, and the character replied about the wrong video. The reader's turns fell into two groups with nothing between 1,500 and 3,000 characters: remarks (the median was 122) and pasted posts (all over 3,000). A turn in the second group asks the memory nothing, so it is not asked, and no room is set aside for memories that turn.
- **At most `recall_count` turns** (4), most relevant first. Then chapter 5's builder applies the real limit — the recall *share* of the budget — and puts the survivors back in story order.
- **Any failure costs the reader nothing but the memories.** The embedding host down, a wrong dimension, a timeout: the reply goes out without recalled turns. airp had a bug here — an embedding *timeout* surfaced as a cancellation and failed the whole turn — so the test for it is explicit.
- **Embeddings are not in the ledger.** The whole corpus of a story costs under a cent; airp's cost report said so rather than pretend to completeness, and so does this one.

> **WORTH KNOWING**
>
> DeepSeek has no embeddings endpoint, so the embedding model stays on OpenRouter (`openai/text-embedding-3-small`) whatever writes the replies. Setting

`LUSTJINN_EMBEDDING_MODEL` to empty switches retrieval off entirely — summaries and facts keep working.

6.5 Background calls: once more, only if it could help

Summaries and extractions are the story's own calls, with nobody waiting on them. A refusing summariser is a character that forgets, so they are asked once more — but only for a failure a second attempt could fix:

```
def worth_another_go(error: ModelError) -> bool:
    status = error.status
    if status is None:  # a timeout, a dropped connection, a body that was not JSON
        return True
    return status in RETRYABLE_STATUSES or status >= 500   # 200 with nothing in it, 408, 429
```

A 401 or a 404 is not retried: a rejected key is rejected twice, and the second call would bill for the same refusal. Two failures running give up. **A reply to the reader is never retried here**: the reader sees the failure and decides.

> **THE .NET LENS**
>
> `once_more_if_worth_it[T](call: Callable[[], Awaitable[T]]) -> T` is a generic method taking a `Func<Task<T>>` — Python 3.12's syntax for type parameters on a function, `def f[T](...)`, is new enough that many examples online still use `TypeVar`. The policy itself is a one-retry Polly policy with a predicate; written out, it is six lines.

6.6 Testing the memory

Every memory test plays on the dummy character. That is not a convenience: airp's memory tests set a four-token card inline, and a bug that only showed with a real-size card — the summariser not counting the character file when deciding what fit — cost a real story twenty-four turns of memory. Here the fixed layers are the size they will be in production, and the budget binds the way it will.

The scripted model from chapter 3 grew three verbs for this step:

- `summarises(gist)` answers a summary padded past the credibility floor — a test that wants a *refused* summary scripts `says("##")` instead.
- `extracts(facts, retired)` answers the extractor's JSON, optionally in a code fence.
- **The embeddings endpoint** answers with a *keyword embedder*: a text's vector says which of four words it contains, so "the knife" and "a knife on the table" are near, and two unrelated texts are orthogonal rather than identical. Deterministic, and no network.

And a `tune()` fixture changes a setting for one test — `tune(context_budget=…)` — so a story of thirty turns can be made to overflow without being three hundred turns long.

> **WATCH OUT**
>
> Every summary is followed by an extraction. When the summariser was first wired in, nine summary tests broke at once: each had scripted a summary and a reply, and the extractor

quietly ate the reply. The fix was one line per test (`.summarises(…).extracts().says(…)`), but the lesson is the one from chapter 3's spend ledger: a call that fires on its own is a call that is easy to forget — in the budget, in the bill, and in the tests.

TRY IT

Make the memory work in front of you. Set a small budget so compression starts early:

```
LUSTJINN_CONTEXT_BUDGET=6000
```

Run `uv run python scripts/seed_dummy.py` once more if you have not since chapter 4, so the dummy persona is the default. Restart the server, start a story on the dummy character and play fifteen or twenty short turns. Then look at what happened:

```
SELECT from_sequence, to_sequence, message_count, left(text, 80) FROM summaries;
SELECT subject, text, valid_from_sequence, valid_to_sequence FROM facts ORDER BY created_at;
SELECT count(*) FROM embeddings;
SELECT kind, count(*), sum(cost) FROM spend GROUP BY kind;
```

Then ask the character about something from the first few turns — a name, an object — and read `GET /stories/{id}/audit`: the newest reply's `context` line names a `memories` layer, and its `summaries` and `world` layers carry the tokens that replaced the turns you no longer see. Put the budget back.

6.7 A real run

Before this chapter was finished, the memory played a real story: twelve short turns on the dummy character, then a question about the first night, with the budget forced down to 6,000 tokens so that compression would start within the hour instead of within the week. DeepSeek V4 Flash wrote the replies and the summaries; `text-embedding-3-small` made the vectors. What the tables held afterwards:

What	Count	Cost
Replies	13	$0.0086
Summaries	5	$0.0024
Fact extractions	4 billed (one failed, cost nothing)	$0.0029
Facts kept	21, none retired yet	
Embeddings	21 at the time; the five turns just compressed join on the next turn	

The first summary covered turns 1–10, as designed. The next four covered 4, 5, 2 and 5 turns: with a budget that tight, fewer than ten turns are ever to spare beyond the newest six, and the batch shrinks to what there is. At 32,000 the batches stay at ten. The facts were the right kind — *Rowan Hale is a surveyor for the Meridian Assurance Company* from turn 1, *Mags deals left-handed* from turn 6 — and

included details the character had invented itself, such as which way the bolt on room seven throws, which is exactly what facts are for: once said, it is true. The estimate for the last prompt was 4,716 tokens; DeepSeek billed 4,720.

And the run found the bug described above under `ALWAYS_WHOLE`: the thirteenth turn, the question itself, was summarised, and the audit line read `character 2360 · world 417 · summaries 1939 · total 4716/6000` — no history. After the fix the same question gave `history 32 (1 dropped) · memories 530 · total 5278/6000`: the question kept, the previous reply dropped by the builder because nothing else could give, and five recalled turns brought back for it.

One more thing it showed. The story had been created without a persona, and the local database had no default persona set — it was seeded before the default existed — so the recalled turns named the reader *the reader*. The rule held (never "User"), but a story with nobody to play as is a weaker prompt than it needs to be. The try box below starts by making sure the seed has run.

6.8 What step 6 leaves behind

- A story can run past its budget without forgetting: summaries carry what happened, facts carry what is true, retrieval brings back what was said.
- Nothing fires for a story that fits, and nothing that fires can cost the reader their reply.
- Three new tables (`summaries`, `embeddings`, `facts`), pgvector enabled, and a ledger that shows every background call beside the replies.

Next — Chapter 7, Story features. What airp does beyond the core loop, as API features both clients will use: directions with `/do`, dials, meters, regenerating with a reason, branching a story, search, export, cost reports, editing facts by hand, and rebuilding a story's memory from its transcript.

CHAPTER 7

Story features

Step 7 adds no new mechanism. Everything the memory needed is in place; what this step adds is what airp does *beyond* the core loop — eighteen features, each a few rules on top of chapters 2 to 6 — and the thing it teaches is how an API grows feature by feature without growing a tangle. It shipped as two pull requests: the features that steer a turn, then the features that manage a story, with this chapter.

Steering the turn	Managing the story
dials, meters	branch, delete from a message
carry on, `/do`, `/focus`	search, export
regenerate with a reason	cost reports
a model per story — taken out again	facts by hand
`/recap`	rebuild the memory, purge

7.1 One module per feature

Every feature here is one file under `src/lustjinn/` with the same shape: pure functions at the top, the few `async def` that touch the database under them, and a thin `APIRouter` at the bottom. `main.py` adds one line per module, behind the sign-in guard:

```
app.include_router(dials.router, dependencies=[Depends(auth.require_user)])
app.include_router(trackers.router, dependencies=[Depends(auth.require_user)])
app.include_router(editing.router, dependencies=[Depends(auth.require_user)])
```

A router with a prefix (`/stories/{story_id}/trackers`) is the feature's own namespace; the `story_id` in the prefix becomes a parameter of every handler under it. The one thing they all share is `stories.visible_story(session, story_id)` : the story, or a 404 that does not say whether it was deleted or never existed.

> **THE .NET LENS**
>
> An `APIRouter` is an ASP.NET Core controller, or a minimal-API `MapGroup` , with the route prefix and the `[Authorize]` attribute applied where it is *included* rather than where it is *declared*. The module-per-feature layout is the vertical slice: the request model, the rules and the handler for

> one feature next to each other, instead of a Controllers folder, a Services folder and a Models folder each holding a slice of every feature.

The rule had one exception for a while: `turns.py`. Every command `/send` dispatches grew inside it — `/ask` and its list, `/recap`, `/fact`, `/tracker`, and the audit — until it was 725 lines that every change touched (#133). They now live with their nouns: `asides.py` (new), `audit.py` (new), `recap.py`, `facts.py`, `trackers.py`, and `turns.py` keeps send, carry on and reroll, at 403 lines. The split had one trap. `/send` imports the modules it dispatches to, so none of them may import `turns` back — Python resolves imports as it runs them, and a cycle fails at start-up with half a module defined. What they shared moved *down* instead: the events and the streaming response into `streams.py`, and each helper `turns.py` had kept private to the module of its noun — the loaded story and its visible messages to `stories`, the persona in force to `library`, a story's dials to `dials`.

> **THE .NET LENS**
> C# resolves types across a whole assembly at compile time, so two classes may reference each other freely; Python modules are objects built top to bottom when first imported, and `a` importing `b` importing `a` finds `a` half-built. The cure is the one you would use to break a project-reference cycle: move what both need into a third module that depends on neither.

7.2 Dials: a pack of data

"Be more varied", written into a prompt, is the weakest way to move a model. Temperature moves it; so does the token ceiling. So each dial declares its **lever**: `prompt` puts the chosen text in the directives layer, `sampler` sets an API parameter and injects nothing, `both` does one of each per level. The sixteen shipped dials are a JSON file inside the package — Render has no disk to put a reader's own file on — read once and validated **whole**:

```
@lru_cache
def shipped() -> Pack:
    return parse(resources.files("lustjinn").joinpath("dials.json").read_text(encoding="utf-8"))
```

A scale has exactly five levels, read by index — a four-level scale would make the top of the dial quietly mean the bottom — a sampler lever names its parameter, a template carries its placeholder, or the dial is skipped with its reason recorded. A story stores only the values it changed (`dial_values`, one row per dial), and the value in force is:

```
def effective(dial: Dial, values: Mapping[str, str]) -> str | None:
    stored = values.get(dial.key)
    return stored if dial.enabled and stored is not None else dial.default
```

Disabled is *pinned*, not off: the default still applies on every prompt, and a stored value survives to resurface the day the dial is re-enabled. Rendering walks the pack in order — the one-line dials as a block, the paragraph-shaped ones after — which is cache-stable until a dial moves:

```
match dial.kind:
    case Kind.SCALE if (index := level_index(dial, value)) is not None:
        level = dial.levels[index]
```

```
            lines.append(f"{dial.title}: {level.label} — {level.text}.")
        case Kind.TOGGLE if is_on(value) and dial.on_text:
            blocks.append(dial.on_text.strip())
        case _:
            pass  # a value the pack cannot read says nothing rather than something wrong
```

> **THE .NET LENS**
>
> `match … case Kind.SCALE if …` is a `switch` expression with a `when` guard; the walrus `(index := …)` binds inside the guard the way a pattern's designation does. `importlib.resources` is `Assembly.GetManifestResourceStream` — a file that ships inside the package and is read by name, not by path. `StrEnum` is an enum whose members *are* their strings, which is what you want when the values live in JSON and in a database column.

Two decisions worth knowing. Summaries stay at 0.3 whatever Creativity says. And the response-length dial's ceiling is also the room the summariser reserves (`reply_tokens`), so the memory and the builder agree about how much room the transcript has — the donor left that one open, and two components disagreeing about room is how it once lost twenty-four turns.

7.2.1 Saying what the dials are

The rendered lines first went out bare, as airp sent them: `Lust: Unhinged — no limits, no pacing, maximum intensity every message.`, right after a card thousands of tokens long. Nothing said whether that was an order, a mood or a note, nor which side wins when the card describes someone reserved. The model split the difference, and the card mostly won — where the reader expected what other apps do, a character who *adapts* to the dial in their own way. So the layer now goes under a frame, as the persona always has (#142):

```
DIRECTIVES_FRAME = (
    "The reader has set how this story is told. These settings outrank anything in the "
    "character's description that disagrees with them. The character stays who they are, in "
    "voice, history and manner, and acts the settings out through that: a shy character at the "
    "highest heat is shy and losing that fight, not suddenly someone else.\n\n"
)
```

It is chapter 3's rule again — every instruction sent to the model says what it is — applied to the one layer that had escaped it. The frame is constant text, so it costs sixty-seven tokens a turn and nothing in cache: it changes only when a dial does, which is when the layer changed anyway. It is sent only when a dial says something; a story with no dials set gets no frame.

> **THE .NET LENS**
>
> `layers.directives and DIRECTIVES_FRAME + layers.directives` is a null-and-empty guard in one expression: `and` returns its left side when that is falsy (`None`, `""`) and its right side otherwise — `string.IsNullOrEmpty(d) ? d : Frame + d` in C#. Python's `and` / `or` return an operand, not a `bool`, which is why the idiom works.

7.3 Meters: a value the model draws and the app reads back

A meter is a number the story keeps — trust, heat, suspicion — with a name, a range and, if the reader wants, what it measures, what the numbers mean and what constrains it. Each turn the stored

value is rendered into the prompt with the shape the model must draw it back in, the model ends its reply with the meter moved, and the line is read back:

```
RENDERED = re.compile(
    r"\[\s*(?P<name>[^\]\n]{1,120}?)\s*\]\s*(?P<bar>[^\n\d]*?)\s*"
    r"(?P<value>-?\d+(?:\.\d+)?)\s*/\s*(?P<max>\d+(?:\.\d+)?)\s*\|\s*"
    r"(?:Δ|delta)?\s*(?P<delta>[+\-]?\d+(?:\.\d+)?)\s*\|\s*(?P<note>[^\n|]{0,200})",
    re.IGNORECASE,
)
```

Deliberately loose about the bar — models draw hearts, blocks or nothing — and strict about three things: only a meter the story *has* is touched (a model that invents one is writing fiction, not configuration), the value is clamped to the range rather than trusted, and the **delta is computed, never believed**. That round trip is what keeps a meter honest three hundred turns later, when the turn that last drew it has been compressed away: the value lives in the table, not in the transcript.

> **WATCH OUT**
> A meter the model can see is a meter it writes towards. Meters are off by default for that reason, and the drawn lines stay in the stored reply as the donor left them — they reach history, summaries and embeddings. Stripping them would be a change to a reply the model wrote.

`/tracker <name> <value>` moves one by hand; the value is the last word, so a two-word name needs no quoting. Python's `rpartition(" ")` does the split in one call.

7.4 Directions: turns the reader asks for without writing one

Three commands call the model with nothing from the reader. **Carry on** (`POST /stories/{id}/continue`) sends the donor's directive that the reader's silence is not a cue to stop. `/do <direction>` frames a direction; alone it is a turn under that direction, over a message (after a blank line) the message is stored and the direction steers the reply. `/focus <who>` hands the turn to a named character. None of them puts a word into `messages`: the direction is the prompt's *last layer*, and a direction replaces the carry-on wording rather than joining it — two answers to "what should this turn be" in one prompt is how a reply comes back trying to satisfy neither.

A direction is part of what is asked for, so it is part of the request's identity: `request_hash(story_id, anchor, text, instruction)` — the same words under a new direction are a new ask, and a retry under the same one finds the stored message.

One small trick made the commands that answer *without* a model call fit the streaming endpoint. `/recap`, `/tracker` and `/fact` answer in a single `said` event, so a client has one code path for everything `/send` returns; but a refusal (a word where a number should be) must be a 4xx, not a stream of one. So the generator is run up to its first event *before* the response opens (`streams.started`):

```
async def started(events: AsyncIterator[str]) -> AsyncIterator[str]:
    first = await anext(events)  # a refusal raises here, as an HTTPException

    async def rest() -> AsyncIterator[str]:
        yield first
        async for event in events:
            yield event
```

```
    return rest()
```

> **THE .NET LENS**
>
> An `async def` with `yield` is an `IAsyncEnumerable<T>` method, and `anext()` is `await enumerator.MoveNextAsync()` for the first item. The pattern — pull one item to surface early failures, then hand the rest on — is the same one you would write to make a streaming action return `400` instead of a broken stream.

7.5 Regenerate with a reason

The reason is the only thing that makes the second attempt differ from the first. Nine reasons, nine notes under one frame, and a rule enforced as written: **no directive may contain "previous reply"**, because the withdrawn attempt is hidden from the prompt before the call and the model cannot see what it is being asked to differ from. The reader's own words are framed as a direction under the note, never read as the latest message.

```
@router.post("/{story_id}/reroll", responses=STREAMED)
async def reroll(story_id: uuid.UUID, ..., body: Reroll | None = None) -> StreamingResponse:
```

A body typed `Reroll | None = None` is optional on a POST: the chapter 4 clients that send nothing still work, and one that sends `{"reason": "looping"}` gets its note.

7.6 A model per story — built, then taken out

Step 7 shipped a model per story, ported from airp: a story could name its own model, checked against the provider's list before it was saved; its budget was fitted to that model's window; its temperature was mapped onto the model's range; and the default wrote any turn the story's model could not take, with the reply saying so.

It was taken out again (#140), once airp had shown what the choice was worth. The model billed as the most explicit wrote about like the default, and the larger ones cost several times as much for no difference a reader could name. What changes a reply is the card, the dials and the host serving the model — all of which work on the default. Every rule the choice needed (a temperature map, window corrections, a fitted budget, a fallback, a cached model list) existed only to keep the choice from breaking a story. Every story now plays on `LUSTJINN_MODEL`; changing it is one variable.

Taking a feature out is a migration like any other, read backwards:

```
def upgrade() -> None:
    op.drop_column("messages", "fell_back_from")
    op.drop_column("stories", "model_context")
    op.drop_column("stories", "model")


def downgrade() -> None:
    op.add_column("stories", sa.Column("model", sa.String(length=200), nullable=True))
    op.add_column("stories", sa.Column("model_context", sa.Integer(), nullable=True))
    op.add_column("messages", sa.Column("fell_back_from", sa.String(length=200), nullable=True))
```

The downgrade brings the columns back empty, not their values: a dropped column's data is gone. That is why the data was checked first — no story on staging had named a model, and no reply had ever fallen back.

> **THE .NET LENS**
>
> `op.drop_column` is EF Core's `migrationBuilder.DropColumn`, and `downgrade` is `Down`. Deleting the attributes from the SQLAlchemy model without a migration fails the schema test from chapter 2 — the models and the migrations must agree — exactly as an EF model change without a migration leaves `PendingModelChangesWarning`. An API field removed from a Pydantic body is ignored, not refused, when an old client still sends it: `BaseModel` drops unknown fields by default, as `System.Text.Json` does.

7.7 Branch, and delete from a message

Both edit a story's shape without rewriting a row. **Branch** (`POST /stories/{id}/branch`) copies the story up to a chosen turn under a new name, with the memory those turns built, by four rules worth stating because each one is a wrong answer avoided:

- live messages keep their sequence numbers, and their **embeddings are carried**, not recomputed — the text is identical, so the vector is too, and retrieval in the branch works from turn one;
- only **summaries wholly inside** the branch: one that straddles the point describes turns the copy does not have;
- **facts true at the point**: a fact retired *after* it was retired by turns this copy lacks, so here it never stopped being true (`valid_to_sequence = None`);
- **meters at the value they hold now**, the one thing that cannot be rewound; dial choices as they stand; and never spend — the branch starts at zero.

Delete from (`DELETE /stories/{id}/messages/{message_id}`) hides the turn and everything after it; the rows stay, as the trigger insists. What to do with the memory of the hidden turns was a decision the donor left open, and it is the branch rules run backwards: a summary that reaches the cut goes, a fact extracted at or after it goes (a pinned one stays — a person said it), and a fact those turns had retired is true again. The next sequence number is never reused.

7.8 Search, export, cost

Search (`GET /search?q=`) is a case-insensitive *phrase* search over visible turns of visible stories — `ILIKE` with the pattern's own `%` and `_` escaped — because the question is almost always "where were these words said", and stemming would answer a different one. Postgres full-text search was weighed and set aside: one reader's stories are small enough to scan. Story names score 60, turns 40 plus five per further occurrence up to twenty, newest first among equals; inside one story the excerpt is longer.

Export (`GET /stories/{id}/export?format=`) is one projection rendered three ways — Markdown with front matter and a heading per turn, JSON with one entry per turn and a resolved speaker, plain text with a numbered stamp line — and never the hidden turns. The response carries a `Content-Disposition` with a file name; each client saves or copies it.

Cost reports (`GET /spend`, `GET /stories/{id}/spend`) are sums over the ledger from chapter 3, read at report time: by kind, by story (dearest first; an erased story under "(purged)") and by provider, with the cached share and what went on replies since rerolled or cut away. Two rules the tests pin against hand-made totals: **a call with no price is counted as unpriced, never as zero**, and the window is `[from_at, to_at)`. Embeddings are not in the ledger and are not counted, and the report says so.

> **THE .NET LENS**
>
> The tally is a mutable `@dataclass` with a method, built up in one pass over the rows — the Python equivalent of a `GroupBy` followed by `Aggregate`, written out because the same row feeds five tallies (the whole, its kind, its story, its story's kind, its provider). `Decimal` sums stay exact; the JSON carries them as strings, which is why the tests wrap them in `Decimal()`.

7.9 Facts by hand

A wrong fact is injected into every prompt until something contradicts it — and since the character acts on it, nothing does. So `GET/POST/DELETE /stories/{id}/facts` let a person see what the story believes, state a fact (pinned from the current turn, so the extractor cannot retire it) and retire one — pinned or not; a person can do what the extractor cannot. `/fact <statement>` pins under the character's name without a model call, and an `/ask` answer worth keeping is one `POST` away from becoming something the character believes.

7.10 Rebuild, and purge

The memory is derived and the transcript is never deleted, so **memory made badly by an earlier version can be made again**. `POST /stories/{id}/memory/rebuild` throws away the summaries and the extracted facts and replays `memory.compose` — the same function a turn calls, dials and meters and fitted budget included, retrieval off since nothing is asked — until a pass writes nothing:

```
for _ in range(MOST_PASSES):
    before = await _summaries(session, story.id)
    await memory.compose(..., retrieval=False)
    if await _summaries(session, story.id) == before:
        break
```

A rebuild that used rules of its own would produce a memory the application never would. The bound is the transcript itself; the loop only ends early if something is wrong, and then the report says what it managed. Pinned facts stay. The old calls stay in the ledger and the new ones are added beside them. A dry run only says what it would replace.

Purge (`POST /stories/{id}/purge`) is the real erasure: delete only hides. It erases a story *already deleted* — two deliberate acts, not one — with every row that carries its text, in one transaction, on the one path where chapter 2's append-only trigger stands aside:

```
await session.execute(text("SELECT set_config('lustjinn.purging', 'on', true)"))
```

The third argument makes the setting local to the transaction; the trigger reads it and lets the `DELETE` through, and still refuses an `UPDATE` of a message's text. The spend rows stay — they carry no text,

the money left the account whatever became of the story, and dropping them would make every report for that month wrong.

7.11 Testing eighteen features

Nothing new in kind, and one habit worth naming: **a fixture that builds a story with everything in it**. `tests/test_branching.py` makes the dummy story with a summary, a straddling summary, facts of every kind, embeddings, a meter, a dial, two questions, a hidden reply and a bill — and branch, delete-from and purge all test against it, each asserting the rules on the same rows. Cost reports are tested against totals worked out by hand in the test file, including an unpriced call and a purged story.

> **WORTH KNOWING**
>
> Step 7 made no real calls. Dials and meters are deterministic through the API, and the one thing worth watching against a real model — whether it draws the meter line back in the shape asked — is best seen in a client, which the next two steps build.

> **TRY IT**
>
> Set a dial and watch it reach the request: `PUT /stories/{id}/dials/creativity` with `{"value": "4"}`, send a turn, then read `GET /stories/{id}/audit` — the `directives` layer is in the context line only for prompt levers, and the ledger row shows the model that answered.
>
> Add a meter — `POST /stories/{id}/trackers` with `{"name": "Trust", "value": 40}` — play three turns and read it back: the value moved by what the model drew, the note in its words, the delta computed. Then `/recap` to see the story so far for free, and `GET /stories/{id}/spend` to see what the three turns cost, discarded replies apart.

7.12 What step 7 leaves behind

- An API that does everything airp's two clients did, feature by feature, each in its own module behind the guard.
- Two more tables (`dial_values`, `trackers`) and no new mechanism. (Two columns came and went with the model per story.)
- Every endpoint and all six slash commands tested, so the clients can be built against a contract that does not move.

Next — Chapter 8, The SvelteKit PWA. A design system in dark and light, approved by the owner before the screens are built: sign-in, the stories list, the conversation with its composer and commands, dials, meters, the library, and the audit — all against this API.

CHAPTER 8

The SvelteKit PWA

Seven chapters of backend, and not one screen. This chapter is the first client: a web app you install on a phone, built with **SvelteKit 5**, served as static files, talking to the API over HTTPS and nothing else. It is also the first chapter where the owner approved a design before a line of it was built, and will approve the screens again before the step closes — the rule is written on GitHub #75, and it held.

The step shipped as two pull requests: the shell, sign-in and the two core screens; then the dials and meters, the composer's helpers, the library editor, the Render site and this chapter.

8.1 Why Svelte, and why static

The kickoff chose SvelteKit over React because a static export of Svelte is close to plain HTML with a small runtime, and the app had to be light on a phone. Two decisions followed from it:

- **Static, as a single-page app.** `adapter-static` writes `build/` with one `index.html` that answers every path (`fallback: 'index.html'`), and the root layout turns server rendering off:

  ```ts
  // src/routes/+layout.ts
  export const ssr = false;
  export const prerender = false;
  ```

  So `/story/<id>` opens on a reload, there is no Node server to host, and the site can live on Render's static tier, which never sleeps.

- **Svelte 5 runes from the first line.** `$state`, `$derived`, `$effect`, `$props`. The project forces runes mode in `vite.config.ts` so a Svelte 4 example pasted in fails loudly rather than working by accident — most of what is online is still Svelte 4.

> **THE .NET LENS**
>
> If Blazor is the nearest thing you know: a `.svelte` file is a Razor component — markup, a script block, scoped styles — but it compiles to a small amount of JavaScript at build time; there is no runtime diffing a virtual DOM and no WebAssembly download. `$state` is a property that notifies on set, `$derived` a computed property with its dependencies tracked for you, `$effect` what you would put in `OnParametersSet`. Routes are files: `src/routes/story/[id]/+page.svelte` is `@page "/story/{id}"`.

8.2 The shell: tokens, theme, and a client

8.2.1 Tokens in one stylesheet

Everything the owner approved on the design page became `web/src/lib/theme.css`: the navy ground, the cream ink, the gold that means *you*, and every derived surface and line — defined once for the dark theme on `:root`, again for light under `prefers-color-scheme: light`, and again under `[data-theme='light']` so a choice beats the system. No component names a colour of its own; it says `var(--gold-text)`, and the theme decides what that is.

The theme store is a class with one rune field:

```
class Theme {
    mode = $state<ThemeMode>(remembered());
    set(mode: ThemeMode) {
        this.mode = mode;
        if (mode === 'system') delete document.documentElement.dataset.theme;
        else document.documentElement.dataset.theme = mode;
        localStorage.setItem(KEY, mode); // in a try: a private window has no storage
    }
}
export const theme = new Theme();
```

A `.svelte.ts` file may use runes outside a component, which is how one store serves every screen. One more line matters: `app.html` carries a tiny inline script that applies the remembered choice *before* the first paint, so a light-theme reader never sees a navy flash.

> **THE .NET LENS**
>
> A module that exports `new Theme()` is a singleton service registered once; the screens import it instead of injecting it. `localStorage` is per-origin browser storage, the nearest thing to `ApplicationData.LocalSettings`, and it can be absent or throw, hence every read in a `try`.

8.2.2 The API, typed, and a stream read by hand

`web/src/lib/api.ts` is the one place the API's shapes are spelled out in TypeScript — `Story`, `Message`, `Done`, `Dial`, `Tracker` — and the one place the bearer token is attached. A refusal becomes an `ApiError` with the API's own sentence; a server that cannot be reached becomes `Unreachable`, which is a different thing to the screens.

The turn is the interesting call. The API streams a reply as server-sent events, and the browser's `EventSource` only does GET; a turn is a POST. So the body is read as it arrives and cut into events by hand:

```
export class SseParser {
    private buffer = '';
    push(chunk: string): SseEvent[] {
        this.buffer += chunk.replace(/\r\n/g, '\n');
        const events: SseEvent[] = [];
        let at: number;
        while ((at = this.buffer.indexOf('\n\n')) >= 0) {
            const block = this.buffer.slice(0, at);
            this.buffer = this.buffer.slice(at + 2);
            const parsed = parseBlock(block);
            if (parsed) events.push(parsed);
```

```
        }
        return events;
    }
}
```

A chunk may end mid-line, so the tail is kept between calls; the parser is pure and unit-tested with awkward chunking, CRLF line endings and keep-alive comments. `api.send` feeds each `delta` to a callback and resolves with the `done` or `error` event, typed as a union the screen switches on.

> **THE .NET LENS**
> `SseParser` is what you would write around a `StreamReader` if `HttpClient` gave you the response body as it arrived — which it does, with `HttpCompletionOption.ResponseHeadersRead`. The `Done` union is a discriminated union on `kind`, the TypeScript way of saying `OneOf<Turn, Aside, Said, Error>`; `switch (done.kind)` narrows the type in each case.

8.2.3 The service worker, and a lesson

An installable app needs a manifest and a service worker. The worker caches the built files on install, so the installed app opens instantly offline to its shell; API calls never pass through it. SvelteKit 3 wants the worker in its own folder with its own `tsconfig.json` extending `$app/tsconfig/service-worker`, and gives it `immutable` and `assets` from `$app/manifest`.

The lesson cost an hour: the worker was being registered in the dev server too, and after the dev server restarted it kept serving a cached shell and answering `503 Offline` for modules that had moved. The layout now registers it itself, and only in a production build:

```
if (!dev && 'serviceWorker' in navigator) {
    void navigator.serviceWorker.register('/service-worker.js', { type: 'module' });
}
```

> **WATCH OUT**
> A file whose name contains `.server.` is server-only to SvelteKit — it refuses to bundle it for the browser "as this could leak sensitive information". The store that knocks on `/health` was first called `server.svelte.ts` and the whole app went blank with a clear error in the dev server's log. It is `health.svelte.ts` now.

8.2.4 Icons from the logo

`web/scripts/icons.mjs` renders the manifest icons, a maskable one with the safe zone respected, the Apple touch icon and a favicon from `brand/logo.svg`, with `sharp` as a dev dependency; the outputs are committed so a build needs no rasteriser. The same tile is the header's mark, the lamp on the waking screen, and the cover of this book.

8.3 The gate: waking, then signing in

The root layout renders exactly one of three things:

```
{#if server.state !== 'up'}
    <Waking />
{:else if !session.signedIn}
    <SignIn />
{:else}
    {@render children()}
{/if}
```

The static site opens instantly; the API on Render's free tier sleeps and takes up to a minute to wake. So the first thing the app does is knock on `/health`, and show the lamp breathing until something answers — with the wait growing from two to six seconds, and after two minutes a plainer message and a *try again now*. Sign-in exchanges the name and password for a token the device keeps for a month. A `401` anywhere sends the reader back to the sign-in screen with a note saying why; a call that cannot reach the server sends them back to the lamp. The two stores learn about this through two hooks the API client exposes (`whenSignedOut`, `whenUnreachable`), which keeps `api.ts` free of imports from the stores that import it.

8.4 The two screens the feel lives in

8.4.1 The stories

Newest first, the character's initial, the last line in the book face, the month's spend in the bar from `GET /spend`. A card swiped left shows *Branch* and *Delete*; the swipe is a few pointer events and a `transform`, and a tap on a swiped card closes it rather than opening the story. Pull down at the top to refresh. The gold button starts a new story: pick a character, a persona (the default preselected), a name.

The search box filters with the donor's fuzzy matcher over the story's name, best match first — the same matcher the terminal uses, ported line for line to `fuzzy.ts` (#135). It first matched substrings of the name, the character and the preview, so the same letters ordered different stories in the two clients. A port drifts unless something holds it, so the web's tests include a table of scores computed by the terminal's `fuzzy.py`, and both must agree to the point. Two details make that possible: the port counts code points with `Array.from(text)`, not UTF-16 units, so an emoji in a name costs one character in both languages, and the integer mean of a multi-word query floors as Python's `//` does.

8.4.2 The conversation

The screen you live in. Your turns are gold-edged on the right; the character's are on the left as **runs** — the donor's `ProseFormat` rules ported to `prose.ts` and unit-tested: `*action*` dimmed and italic, `**emphasis**` in gold, `"dialogue"` upright, a star beside whitespace left as a star, a quote that never closes left as a quote. The reply streams in with a gold caret; the reader's message appears before the API confirms it.

Everything the composer does is in one component. A leading `/` opens the command palette from `GET /commands` — a new endpoint, so neither client keeps a copy of the list the API enforces — with what each command costs; Tab completes. A `:name` at a word start opens the picker: the library's snippets by name, inserted as their text, and emoji by name or keyword from the donor's table of 225. `:wave:` becomes 👋 before the message is sent; a bare `:storm` is a snippet trigger the API expands when it stores the message, as chapter 4 built. Enter sends; when the composer is empty the send button reads *Carry on ›*.

The emoji table was first ported twice, once into each client, and two copies of a table are two places to forget an emoji. It is now data in the API's package, `emoji.json` beside `dials.json`, served as `GET /emoji` (#132): the one route besides `/health` and sign-in that answers without a token — it says nothing about anyone's stories — and the one response a browser may cache, for a day. Each client reads it once per session; until then a `:name:` stays as typed, which is what it would be anyway for a name the table lacks.

Every other response goes out `Cache-Control: no-store`, set by a middleware on the way out. For `/emoji` to keep its own say, the middleware sets each private header only if the route did not:

```
for name, value in PRIVATE.items():
    response.headers.setdefault(name, value)
```

> **THE .NET LENS**
> The middleware is `app.Use(async (ctx, next) => { ctx.Response.OnStarting(...); await next(); })` adding headers, and `setdefault` is `TryAdd` on `IHeaderDictionary` where `update` was the indexer: the route's own `Cache-Control`, like `[ResponseCache(Duration = 86400)]`, now wins. A test asserts that `/health` still goes out `no-store`, so the exception stays one route wide.

Answers that are shown once and stored nowhere — `/ask`, `/recap`, a `/tracker` set — appear as dashed notes in the conversation, labelled so. Reroll opens a sheet of the nine reasons; a long press on any turn opens *branch from here*, *cut the story back to here* (with the count of turns it would hide) and *copy*.

8.4.3 Beside it: dials, meters, the prompt

The approved design had a third column on the desktop and a ◐ sheet on the phone. Both render the same component, `StoryRail`, over the same store, `Extras`: the dials in force as controls (five steps for a scale with the level's exact text under it, a switch, options, items, a line), the meters with their bars and deltas (set by hand, remove, add), and the newest reply's audit line read back into bars — `character 2,360 · world 417 …` — beside the story's running cost with rerolled spend apart. A strip of meter pills rides above the composer. After each turn the store refreshes, so a meter moves the moment the reply ends.

> **THE .NET LENS**
> `Extras` is a view model: `$state` fields the view binds to, methods that call the API and update them. `StoryRail` renders it twice — in the column and in the sheet — with `bind:section` so both show the same tab, which is a two-way binding to a prop, Svelte's `@bind-Value`.

8.5 The library editor

Three shelves as tabs, the default persona marked and settable from the list, hidden entries on request. The editor saves with the version it opened, and when the API answers `409` because a save landed in between, it shows their text under yours with *Replace it with mine* or *Take theirs* — nothing is lost either way, which is the whole point of chapter 4's version column. Every saved version is a sheet away and any of them can be put back in the editor.

8.6 Deploying a static site

`render.yaml` gained a second service: a static site built from `web/`, every path rewritten to `index.html`, `noindex` and `no-referrer` headers, the service worker never cached, and the commit written to `/version.txt` by the build command. The API's URL is baked in at build time from `VITE_API_URL`; the API's `LUSTJINN_CORS_ORIGINS` must name the site. That is a two-step dance done once in Render's dashboard, written down in `docs/DEPLOYMENT.md`. CI's deploy input gained `web-staging` and `both`; the web deploy job fires the hook and waits until the site reports the commit. Nothing was triggered: the cloud comes when the owner says so.

8.7 Testing a client

The gates in CI's `web` job: `prettier --check`, `svelte-check` (TypeScript across `.svelte` files too), `vitest`, and the build itself. The unit tests cover the pure modules — the SSE parser, the prose runs, the shortcode scanner, the audit line — because that is where a wrong rule hides; the screens were checked by hand against the running API, with a dev token minted from the settings and put in `localStorage`. Two things learned the hard way: `#lib/*` imports in `.svelte` files carry their `.ts` extension, or svelte-check cannot find them; and a prop cannot be called `state` in a runes file, since `$state` is the rune.

> **TRY IT**
>
> Run the API and the web app side by side — `uv run uvicorn lustjinn.main:app` and, in `web/`, `npm run dev` — with `LUSTJINN_CORS_ORIGINS=["http://localhost:5173"]` in `.env`. Open http://localhost:5173, sign in, and play a turn on the dummy. Then switch the theme on the *You* screen, open the dials with ◐, set Lust to Explicit and watch the next reply's audit line gain a `directives` layer in the rail. On a phone on the same network, `npm run dev -- --host` and *Add to Home Screen*.

8.8 What step 8 leaves behind

- A web app in `web/`: installable, dark and light, phone and desktop, every screen the API has a feature for, approved by the owner before and after it was built.
- A pattern for a client: one typed API module, stores as classes with runes, a component per screen with scoped styles, pure logic in `.ts` files with tests.
- A second Render service described and documented, not yet deployed.

Next — Chapter 9, The terminal client. The same stories in a terminal, with Textual: the look and feel of airp's console client, dark by design, against the same API.

CHAPTER 9

The terminal client

The second client, and the one closest to home: airp's console client was the thing the owner built by hand on Spectre.Console, with its own view stack, key map, legend and theme. This chapter rebuilds it in Python with **Textual**, against the same API the web app uses, dark by design — one theme, no switch, no approval gate. The kickoff's reason for Textual was that it ships what airp's shell wrote itself: a screen stack, bindings with a footer legend, a command palette, themes and mouse support, and it is async, so a streamed reply fits without a thread.

The step is one pull request of ten issues: the skeleton, the API client and the gate, the stories, the conversation, the composer's commands, the library and the settings, the keys that work everywhere, the dialects and the phone, the composer's helpers, and this chapter.

9.1 A second package in the workspace

The client is not a module of the API. It lives in `tui/` as its own package, `lustjinn-tui`, and the root `pyproject.toml` makes the two a **uv workspace**:

```
[tool.uv.workspace]
members = ["tui"]

[tool.uv.sources]
lustjinn-tui = { workspace = true }

[dependency-groups]
dev = ["lustjinn-tui", "pyright>=1.1.414", "pytest>=9.1.1", ...]
```

One lock file and one environment, so `uv sync` installs both and `uv run lustjinn-tui` opens the client. But Textual is a dependency of `tui/pyproject.toml`, not of the API's, and the client is listed only in the `dev` group: Render installs the API without the terminal, and the terminal's tests run under the root `uv run pytest` because `testpaths` names `tui/tests` too. Pyright and ruff cover both packages from the root; isort is told that `lustjinn_tui` and the test helpers are first-party, so the imports group the way they do in the API.

> **THE .NET LENS**
>
> A workspace is a solution with two projects and a shared `Directory.Packages.props`. The `dev` group is the test project's `ProjectReference` to the client: the solution builds it and tests it, but publishing the API project does not carry it along. `uv run lustjinn-tui` is the `[project.scripts]` entry, the `dotnet tool`-style command that `app.py`'s `main()` answers.

9.2 Textual's model, mapped to airp's shell

Textual is an **app** that owns a stack of **screens**, each composed of **widgets**, drawn with a stylesheet (TCSS) and driven by **bindings**, **messages** and **workers**. Every piece of it has a counterpart in `src/Airp.Terminal`, which is how the port was planned.

Airp (C#, Spectre.Console)	Textual	Here
`Shell` — the loop, the view stack, the redraw	`App`, `push_screen` / `pop_screen`	`app.py`
`View` — title, keys, `Render`	`Screen`, `compose`, `BINDINGS`	`view.py`
`KeyMap` — what a key means	`Binding` + a dialect table	`dialect.py`
`Theme` — the Dark palette	a registered `Theme` with variables	`theme.py`
`ListState` — cursor, page, selection	the same class, ported	`listing.py`
`Run(label, work)` — the spinner	`app.call()` round a worker	`app.py`

9.2.1 The app, and the gate

`LustjinnApp` is airp's `Shell`: it owns the API client, the one theme, the dialect and the few bindings that hold on every screen (`Ctrl+C` quits at once, `Ctrl+P` or `:` opens the palette, `Ctrl+F` searches every story). Its default screen is the stories, but the first thing it does is push the lamp on top:

```
def on_mount(self) -> None:
    self.push_screen(WakingScreen())
```

The waking screen knocks on `/health` in a worker with the waits growing from two to six seconds, as the web app does, and after two minutes says so more plainly. When something answers it pops itself and, without a token on disk, pushes the sign-in screen. The app has one method for the error every screen could meet:

```
async def call[T](self, label: str, work: Awaitable[T]) -> T | None:
    line.busy(label)
    try:
        return await work
    except SignedOutError as refused:
        self.signed_out(refused.detail)      # forget the token, ask again, say why
    except UnreachableError:
        self.lost(f"The server stopped answering while {label.lower()}.")
    except ApiError as refused:
        screen.status(refused.detail, Kind.ERROR)
    finally:
        line.idle()
    return None
```

A `401` anywhere becomes the sign-in screen with a note; a server that stops answering becomes the lamp again; a refusal becomes the API's own sentence in red on the status row, and the caller gets `None` and carries on. That is airp's `Run` with the three outcomes airp learned to tell apart.

THE .NET LENS

`async def call[T]` is a generic method, the PEP 695 syntax for `async Task<T?> Call<T>(...)`. Everything in Textual is `async` on one event loop, so there is no `Dispatcher.Invoke` and no `ConfigureAwait`; a screen that `await`s the API simply yields to the loop and the redraw continues around it. The `View.lustjinn` property exists because Textual types `self.app` as `App[object]`, and pyright in strict mode wants to know it is ours.

9.2.2 The view, the frame, and the dialect

Every screen extends `View`, which draws airp's frame around whatever the screen composes: the masthead (the server badge, the breadcrumb of open screens), the body, a hairline, the legend and the status row.

```
def compose(self) -> ComposeResult:
    yield Masthead(badge=self.lustjinn.badge)
    with Vertical(id="body"):
        yield from self.body()
    yield Hairline(id="footer-rule")
    yield Legend()
    yield StatusLine()
```

A screen declares its `TITLE`, its `HINTS` in the donor's order (the legend fits them to the width with airp's rule: every hint but the last leaves thirteen cells for *? All keys*) and its `BINDINGS`. `Esc` is bound here once, to go back one screen and quit from the last. The frame also follows the width live: under sixty columns — a phone held upright, over SSH, at a font you can read — the masthead becomes one row, the legend folds into the status row, and `layout_changed(narrow)` lets a screen lay itself out again. It changes back as soon as the terminal is wider.

The keyboard dialect is applied in the same class. Airp's `KeyMap` was a table per dialect; here the screens bind the arrows and the chords once, and the dialect decides what the letters mean before the bindings see them:

```
def on_key(self, event: events.Key) -> None:
    if self.typing or event.character is None:
        return
    meant = translate(self.lustjinn.dialect, event.key)
    if meant is None:
        return
    event.stop()
    event.prevent_default()
    if meant != SWALLOW:
        self.lustjinn.post_message(events.Key(meant, None))
```

Vim's `h j k l` become the arrows, `G` the end, `u` undo — only while navigating; inside any text field every printable key types itself, so there is no mode to be in. Because `G` is the end in Vim, regenerate is `Ctrl+G` there: a key has to mean one thing whichever dialect is configured, and the help screen says which.

WATCH OUT

A key translated to itself is not a no-op. The first table mapped `N` to `N` in Vim, and the synthetic `Key("N")` the view posted never matched the screen's `n,N` binding, while the real event would

> have. The letters a dialect does not change must be left out of its table, so the real event reaches the screen untouched; the test that presses `N` in Vim and expects the previous match is what caught it.

9.2.3 Widgets, messages and `@on`

A Textual widget draws itself in `render()` and tells its screen what happened by posting a **message**, which bubbles up to the first handler that claims it. The list of stories is one generic widget over the donor's `ListState`:

```
class Rows[T](Widget):
    class Selected(Message):
        def __init__(self, rows: Rows[Any], index: int) -> None:
            super().__init__()
            self.rows = rows
            self.index = index

        @property
        def control(self) -> Rows[Any]:
            return self.rows
```

The screen handles it by name and, when it has several lists, by selector:

```
@on(Rows.Selected, "#stories")
def _selected(self, event: Rows.Selected) -> None: ...
```

The `control` property is what makes the selector work: `@on` filters on the message's `control`, and a message without one matches nothing and raises nothing, which cost an afternoon. Every message in the client carries it.

> **THE .NET LENS**
>
> A `Message` is a routed event: it bubbles from the widget to the screen to the app, and `event.stop()` is `e.Handled = true`. `@on(Rows.Selected, "#stories")` is the XAML `SelectionChanged="..."` on one named control; `on_key` and `on_mount` are the convention-named handlers, `OnKeyDown` and `OnLoaded`. `compose()` is the markup: it yields the children, and `with Vertical(id="body"):` is a `StackPanel` with its children indented beneath it.

9.2.4 Workers: a stream without a thread

Anything that waits on the network runs in a **worker**, so the screen keeps drawing and the keys keep working. A turn is the long one: the composer posts `Send`, the screen starts a worker over the API's event stream, and each event lands in the transcript as it arrives.

```
self._turn = self.run_worker(partial(self._run, label, stream, replacing), exclusive=True)

async def _run(self, label, stream, replacing) -> None:
    transcript.begin_pending()
    try:
        async for event in stream:
            if isinstance(event, Delta):
                transcript.grow_pending(event.text)
            elif isinstance(event, TurnDone):
```

```
                self._arrived(event)
            elif isinstance(event, Failed):
                self._failed(event)
            ...
    finally:
        self._end_turn()
```

`exclusive=True` cancels the previous worker of that group, which is how `Esc` while waiting stops the stream — the message went, the screen says so, and the reply arrives on the next refresh. The stream itself is `httpx2.EventSource` over a streaming POST, each event parsed by pydantic into `Delta`, `TurnDone`, `AsideDone`, `Said` or `Failed`; chapter 8 cut the same stream by hand in TypeScript, and here the library does it.

> **WATCH OUT**
>
> `run_worker` takes a coroutine *or* a callable. Pass the coroutine of a method you also keep a reference to and Python warns that it was never awaited when the exclusive group cancels it before it starts; pass `partial(self._run, ...)` or the bound method and the worker creates the coroutine itself when it runs. The warning is an error in this repo's pytest, which is how it was found.

> **THE .NET LENS**
>
> A worker is `Task.Run` with a cancellation token Textual holds for you; `exclusive=True` is cancelling the previous token before starting the next. `async for event in stream` is `await foreach` over an `IAsyncEnumerable<Event>`; `_stream` in `api.py` is an async generator (`yield` inside `async def`), the Python way of writing one.

9.2.5 TCSS and the one theme

Textual styles widgets with a CSS dialect. Each widget carries its `DEFAULT_CSS`; the layout rules are a few lines each (`View > #body { height: 1fr; }`). Colour never appears in them. Airp's `Theme.cs` named Spectre colours per role; here the roles are a registered Textual `Theme` and its variables:

```
DARK: Final = Theme(
    name=NAME,
    primary="#87d7ff",      # heading
    foreground="#dadada",   # text
    background="#0a0d1b",   # the brand's navy
    success="#00d787",      # the character's name
    variables={"muted": "#808080", "selection-bg": "#87d7ff", ...},
)
KEY: Final = "[$accent on $surface]"
```

A view writes `$muted` in TCSS and `[$muted]` in content markup, or one of the openers `theme.py` exports (`KEY`, `ACTION`, `SELECTION`); it never names a colour, which is the same rule the web app's tokens enforce. The transcript draws its own lines, and the variables have to resolve there too:

```
strips = content.render_strips(
    width, None, self.visual_style,
    RenderOptions(self._get_style, self.styles.get_rules()),
)
```

That is the widget's own style parser, the way Textual draws any widget's content; rendering to Rich segments directly left `$accent` as literal text.

9.3 The screens

9.3.1 The stories, and a new one

Newest first, one row each: the name, the character, the age of the last line, a preview in the donor's prose runs. `/` filters with the fuzzy matcher ported from airp, the match painted in the highlight role; `F2` renames in place; `Delete` asks first on a confirm screen that pops itself before it runs the work, so the status lands on the list underneath. On resume the list reloads quietly whenever the gate is open, which is the one place the app's `gate_open` is asked: signed in, with neither the lamp nor the sign-in screen on the stack.

A new story is two pickers — character, and persona with the default preselected — and a name defaulting to the character and the date; `Enter` on the last picker creates.

9.3.2 The conversation

The screen you live in, and the one with the most donor rules. The transcript is a `ScrollView` that lays the messages out itself: a row per line, a speaker chip in the role's colour on the surface tone, the selected turn marked with ▌ in the accent. Opening a story lands on its newest turn's first row; the arrows step by three rows and the page follows the selection, as `TranscriptScrollingTests` says it must. The column takes the config's share of the width (sixty percent by default, centred, never under forty columns) and on a phone all of it.

`i` or `Enter` writes. The composer is a `TextArea` that posts `Send` on `Enter` and makes a line break on `Alt+Enter`; on a phone the two swap, because a phone keyboard has no `Alt`. A paste with line breaks is text, not a send. The reply streams in under a pending row with a clock in the header; `Esc` keeps the draft, and a failed turn keeps it too. `>` carries on, `g` regenerates after asking for one of the donor's reasons, `b` branches, `Delete` cuts the story back to the selected turn after asking and says how many went, `/` searches within the story with `n`/`N` round the matches, `c` copies the turn, `x` exports.

9.3.3 The composer's commands and helpers

A leading `/` offers the commands from `GET /commands` in a strip under the draft — one row, because seven names in a list cost seven rows of the story. `Tab` accepts and `Enter` is never claimed by the strip: it sends, it spends money, and a key that sends must not quietly mean something else because a popup is showing. The local commands (`/card`, `/persona`, `/facts`, `/trackers`, `/audit`, `/cost`, `/help`) open a pane over the conversation, and `/search` reuses the in-story search; `/ask` opens a pane the reader can pin as a fact; `/recap` comes back in a pane of its own, a `/fact` or `/tracker` written on the status row, and none of them stores a turn. What the parser cannot place — an unknown name, a command missing its argument — is refused on the status row and **nothing is sent**, which the donor's `SlashCommands` tests insisted on: a mistyped command must never become a line of the story.

The terminal keeps no copy of the server's commands. It first shipped one "as the API stood when this was built", for the moment before `GET /commands` answered, and within a week the copy and the server disagreed (#131). Now the list is read once, the first time a story opens, and kept on the app for every story after. Until it has answered, the parser knows only the client's own commands, and a

name it cannot place comes back as `Unread` rather than `Unknown`: refused all the same — it may be a billed command, and a guess would bill a typo — with a message that says the list is not in yet, and a second read started in the background.

The helpers from airp's composer are the last issue: `:name` offers the library's snippets by name and emoji by name or keyword from the donor's table of 225 — the API's, from `GET /emoji`, read the first time a story opens and held for the app (#132) — `:smile:` becomes 😄 as the closing colon lands, a word offers completions after three letters in the reader's own capitalisation, and an emoji is one character to the cursor. That last one is `graphemes.py`, the part of UAX #29 a chat meets — joiners, skin tones, flags, keycaps — because Textual's text area moves by code point and a backspace would otherwise take one member of a family.

9.3.4 The library and the settings

The library is three shelves as tabs over the same `Rows`, the entry's text in a pane beside it. Editing opens the reader's own `$EDITOR` on a temporary file, with `app.suspend()` handing the terminal over and taking it back; the tests hand the app an editor that needs no terminal. A save that meets a `409` shows the conflict screen with their text and *Replace it with mine* or *Take theirs*, chapter 4's version column again. The story's settings are airp's `ChatSettingsView`: every dial of the pack, stepped or typed, staged and applied together, with *Discard* and *Reload*.

9.3.5 Everywhere: the palette, search, export, the phone

`Ctrl+P` or `:` opens Textual's command palette with the current screen's commands before the global ones, through one `Provider`:

```
class LustjinnCommands(Provider):
    async def search(self, query: str) -> Hits:
        matcher = self.matcher(query)
        for command in commands_of(self.screen):
            score = matcher.match(f"{command.name} {command.description}")
            if score > 0:
                yield Hit(score, matcher.highlight(command.name), partial(self._run, command))
```

A colon typed into a text field inserts a colon; the app tells the two apart by the last key it saw. `Ctrl+F` searches every story with a scope that cycles; `x` exports as markdown, JSON or text to the config's directory, never overwriting. On a phone with `mouse = true` the bottom row becomes buttons — back, then what the screen does most — which press the key they stand for, so a button and its key can never drift apart.

9.4 Configuration and the token

The client keeps its own things in the user's config directory: a commented `config.toml` written with the defaults the first time, and the token beside it in a file only the user can read. The server can be given on the command line or in `LUSTJINN_SERVER`, which beats the file. The keyring was the alternative for the token and would have meant a native backend per platform for a secret the API hands out again on the next sign-in.

> **THE .NET LENS**
>
> `config.py` is `appsettings.json` and `IOptions<T>` without the layering machinery: one file, read with the standard library's `tomllib`, into a frozen dataclass; the environment and the

command line applied as explicit `replace()` calls. `platformdirs` answers `Environment.SpecialFolder.LocalApplicationData` on every OS.

9.5 Testing a terminal

Textual ships a **pilot**: `app.run_test()` runs the app headless at a size you choose, and the test presses keys, clicks and waits. Against it stands a fake server — httpx2's `MockTransport` answering the routes the client uses from memory, with switches for the failures the screens must survive (`asleep_for`, `down`, a revoked token, a model that fails). The tests script a server the way the API's tests script a model.

```
async def test_a_lost_server_sends_the_reader_back_to_the_lamp(app, server) -> None:
    async with app.run_test(size=(100, 30)) as pilot:
        await pilot.pause(0.1)
        server.down = True
        await app.call("Refreshing", app.api.stories())
        await pilot.pause()
        assert isinstance(app.screen, WakingScreen)
        server.down = False
        await pilot.pause(0.2)
        assert isinstance(app.screen, StoriesScreen)
```

The donor's suites were ported by name: `KeyHandlingTests`, `NarrowLayoutTests`, `PhoneBarTests`, `NewChatFlowTests`, `TranscriptScrollingTests`, the composer's helper tests. Resizing is `pilot.resize_terminal(59, 30)` and the frame is asserted to change shape, and back at sixty. None of it touches the network, the database or the reader's config directory, so the suite runs in CI without Postgres and in a second or two.

WATCH OUT

Three things the pilot taught. `AUTO_FOCUS = None` still focuses the first field, because `None` falls back to the app's `"*"`; a screen that wants no focus says `""`. A `scroll_to` right after a `virtual_size` change clamps to the *old* maximum, so the transcript remembers where it wants to be and applies it in `call_after_refresh`. And an attribute named `query` or a method named `commands` on a widget silently shadows Textual's own; the client's are `search_query` and `slash`.

THE .NET LENS

The pilot is a `WebApplicationFactory` for a UI: the real app, in process, driven by a test. `MockTransport` is the `HttpMessageHandler` you hand `HttpClient` in a test. The fixture that builds the app for every test is `conftest.py`, the shared `IClassFixture` of pytest, and the `tmp_path` the token store writes to is a per-test directory pytest deletes.

TRY IT

Run the API and the client side by side: `uv run uvicorn lustjinn.main:app` and `uv run lustjinn-tui`. Sign in, open the dummy story, press `i`, type `so :smi` and watch the strip; `Tab`, then `Enter`. Press `?` for every key. Then set `keyboard = "vim"` in the config file the

> first run wrote, restart, and move the list with `j` and `k` — and type a `j` into the filter to see it stay a letter. Narrow the terminal under sixty columns and watch the frame fold.

9.6 What step 9 leaves behind

- A terminal client in `tui/`: airp's look, keys and rules on Textual, every feature the API has, dark by design, tested headless.
- A second package in the workspace, and a pattern for one: its own `pyproject.toml`, the root's gates covering it, its tests collected by the root's pytest.
- Two decisions to note. The `mouse` flag only chooses the phone's button row, since Textual reports the mouse regardless; and `/search` from the composer reuses the in-story search rather than opening a pane.

The roadmap ends here. Two clients, one API, a memory that does not forget. What comes next is the owner's to choose: the cloud, when it is wanted, is described in `docs/DEPLOYMENT.md`.

CHAPTER 10

Persona pieces

The roadmap ended at step 9, and then the owner asked for something the first nine steps made easy: a persona built from parts. Every chat can have its own persona, but writing one by hand means copying three thousand characters of prose and editing them. This chapter is the feature that came after: a pool of **pieces**, a **joiner** that turns a pick into a persona, a way to **extract** the pieces from the personas that already exist, and a picker in each client. It was five issues in one pull request (with a fix to the memory, below), and it taught three things worth a chapter: a **closed vocabulary** (an enum the database also enforces), a **pure function at the centre** with the side effects around it, and a **proof by round trip** that a refactor of data did not break its format.

It also changed one earlier thing. A reader of chapter 6 will find a paragraph there about turns the context builder dropped before any summary covered them. That was found while this feature was being planned, from the same story's diagnostics, and is fixed in `compose`; it belongs to the memory, so it is told there.

10.1 The persona does not change

The first decision was what *not* to build. A persona is a name and a page of text, a story points at it by id, and editing it reaches every story that uses it (chapter 4). The prompt builder takes it as plain text (chapter 5). Everything that touches personas — search, export, the default persona, the version history — would have to change if a persona became a structure.

So it did not. The pieces are **an authoring tool**: they build the text, once, and the text is copied into an ordinary persona. Editing or deleting a piece later changes no persona already made, because no persona refers to a piece.

> **THE .NET LENS**
>
> This is the difference between a *composed* object and an *assembled* one. In .NET terms the persona is a plain entity and the pieces are a builder: `PersonaBuilder.Build(picks)` returns an entity, and nothing keeps a reference to the builder's inputs. The alternative — an entity made of references to pieces — would make every read a join and every edit a change to someone else's prompt.

10.2 A closed list of slots

The owner's six personas share one skeleton: nine paragraphs, always in the same order — identity and look, style, marks, family, work, home, personality, humour, passions. The pieces are slotted by that skeleton, plus the slots the owner added (a vehicle, pets), and **the list of slots is closed**. There are no user-defined categories. A persona has to come out in the card's format, and free text dropped into a persona may be ignored or overused by the model; a fixed list makes every persona the same shape.

A closed list is an enum, and an enum here is two things at once:

```
class PieceSlot(StrEnum):
    NAME = "name"
    LOOK = "look"
    STYLE = "style"
    MARKS = "marks"
    FAMILY = "family"
    WORK = "work"
    HOME = "home"
    VEHICLE = "vehicle"
    PETS = "pets"
    PERSONALITY = "personality"
    HUMOUR = "humour"
    PASSIONS = "passions"
```

The **declaration order is the persona's paragraph order**: a client that lists the slots in this order shows them in the order the text will read. The same list is written a second time, as a `CHECK` constraint, because the database should refuse what the code would never write:

```
class PersonaPiece(LibraryEntry, Base):
    __tablename__ = "persona_pieces"
    __table_args__ = (
        Index("uq_persona_pieces_slot_voice_name_lower",
              "slot", "voice", text("lower(name)"), unique=True),
        CheckConstraint("slot IN ('name', 'look', …, 'passions')", name="slot"),
        CheckConstraint("voice IN ('he', 'she', 'they', 'any')", name="voice"),
        CheckConstraint("(slot = 'name') = (short IS NOT NULL)", name="short_only_for_names"),
    )

    slot: Mapped[PieceSlot] = mapped_column(_stored_as_text(PieceSlot))
    voice: Mapped[Voice] = mapped_column(_stored_as_text(Voice))
    text: Mapped[str] = mapped_column(Text)
    short: Mapped[str | None] = mapped_column(String(100))
```

Three things to read in it. `LibraryEntry` is the mixin from chapter 4: `PersonaPiece` gets its id, its name, its version and its time of last change for nothing. `_stored_as_text` is the enum-as-text helper in `models.py`, so the table holds `'home'`, not a number. And the third `CHECK` is a **biconditional**: `(slot = 'name') = (short IS NOT NULL)` says a piece has a short form *if and only if* it is a name piece. Two booleans compared with `=` is how SQL writes "both or neither". The index is unique per slot, voice *and* name, case-insensitively: "Penthouse" may exist once for a he persona and once for a she persona.

> **THE .NET LENS**

`StrEnum` is an `enum` stored as its string name — `HasConversion<string>()` in EF Core — and the `CHECK` is `HasCheckConstraint`. The migration (`0016_persona_pieces.py`) is the same table in Alembic's vocabulary, and `test_the_migrations_build_exactly_what_the_models_describe` fails when the two disagree.

10.3 The joiner

Everything the feature knows about the persona's shape is one pure function and two constants. `PARAGRAPHS` says which slots write which paragraph:

```
PARAGRAPHS: tuple[tuple[PieceSlot, ...], ...] = (
    (PieceSlot.LOOK,),
    (PieceSlot.STYLE,),
    (PieceSlot.MARKS,),
    (PieceSlot.FAMILY,),
    (PieceSlot.WORK,),
    (PieceSlot.HOME, PieceSlot.VEHICLE, PieceSlot.PETS),
    (PieceSlot.PERSONALITY,),
    (PieceSlot.HUMOUR,),
    (PieceSlot.PASSIONS,),
)
```

Home, vehicle and pets share one paragraph, their sentences joined with a space, because that is where the owner's personas keep them: "He lives in a penthouse. He rides a Royal Enfield." is one paragraph. Every other slot writes paragraphs of its own, and a piece may hold more than one (the work slot does in two of the personas). `assemble` walks the groups:

```
def assemble(picks: Mapping[PieceSlot, Sequence[PersonaPiece]]) -> str:
    names = picks.get(PieceSlot.NAME, ())
    if len(names) > 1:
        raise PieceError("A persona has one name.")
    paragraphs: list[str] = []
    for group in PARAGRAPHS:
        texts = [piece.text.strip()
                 for slot in group for piece in picks.get(slot, ()) if piece.text.strip()]
        if len(group) == 1:
            paragraphs += texts           # one paragraph a piece
        elif texts:
            paragraphs.append(" ".join(texts))   # home, vehicle and pets: one paragraph
    if not paragraphs:
        raise PieceError("Pick at least one piece.")
    text = "\n".join(paragraphs)
    …
```

Empty slots are skipped, which is how the roommate persona — the same person with no home paragraph — is just a pick with no home piece. The order of the *pick* does not matter, only the order of `PARAGRAPHS`. And the function takes ORM objects and touches no session, so its tests build `PersonaPiece(slot=…, text=…)` in memory and call it, with no database.

THE .NET LENS

A static method over an `IReadOnlyDictionary<PieceSlot, IReadOnlyList<PersonaPiece>>`: no service, no interface, no DI. The rule from chapter 7 — pure functions at the top of a module, the

> few `async def` that touch the database under them — is why the joiner is easy to test and why the two clients cannot disagree with it: neither client joins anything.

10.3.1 The name, written twice

Fourteen of the fifty-seven paragraphs in the owner's personas name the persona, and not only the first: "Allan Sanlúcar El Kettani is an athletic young man…" opens the look, and "Allan is the son of…" opens the family paragraph. A piece that hard-codes "Allan" would carry the wrong name into another persona. So pieces write the name as a placeholder, and there are **two**, because the text uses two forms: `{name}` for the full name and `{short}` for the short one it uses after the first mention. The name piece carries both (its text is the full name, its `short` column the short form — the reason for that `CHECK`), and `assemble` fills them in. A piece that asks for a name when none was picked is refused with a sentence that says so.

10.3.2 The voice, and what was not built

Every one of the six personas uses *he* and *his*. Someone will want a female persona, and the tempting answer is placeholders for the pronouns: `{he}`, `{his}`, `{him}`. It does not work. English pronouns are not a substitution: "his" is "her" in "his sister" and "hers" in "the bag is his"; "He's" and "himself" and the capital at the start of a sentence all have their own rules; and a female persona needs different *content* anyway — body, clothes, tattoos.

So each piece carries a **voice**: `he`, `she`, `they`, or `any` for a piece with no pronouns in it. The builder asks for the persona's voice first and offers only the pieces written in it (or in none), and the server refuses a pick that mixes them. A female pool is a set of pieces written in *she*, tagged *she*. There is no conversion step because there is nothing to convert.

10.4 The API, and what it reuses

Seven endpoints under `/persona-pieces`: the slots (so a client does not hard-code them), the pool by slot and voice, create, save, delete, and the two that matter — **preview** and **build**. Both take the same body (a voice, the picked ids by slot, optionally a name) and both run the joiner; preview returns the text and the name the persona would get, and saves nothing. The clients call it on every change, which is why the web and terminal previews cannot differ from what is saved.

Build makes an ordinary persona, and it does so by the library's own path. `create_entry` used to hold the rules for adding a persona to a shelf — a name already there is refused, the first history row is written, the database's `updated_at` is read back. Rather than copy them, they became `add_entry(session, shelf, name, text)`, and both endpoints call it. The commit that did it changed no behaviour and the library's tests did not notice.

The persona's name is the one thing the joiner cannot decide, because names are unique:

```
async def free_name(session: AsyncSession, wanted: str) -> str:
    """`wanted`, or `wanted-2`, `wanted-3`… the first no persona has, in any case."""
```

It is suggested from the name piece as a *slug* — `unicodedata.normalize("NFKD", …)` and the ASCII letters only, so "Allan Sanlúcar El Kettani" becomes `allan-sanlucar-el-kettani`, like the owner's other persona names — and numbered when taken. A name *given* in the request is used as it is and refused with a 409 when taken: a person who typed a name should be told, not renumbered.

The refusals deserve the same attention as the successes. `PieceError` is a plain `ValueError` the pure code raises; the endpoint turns it into a 422 whose body is the sentence. That is the usual split — the domain says what is wrong, the transport decides the status — and it is why a screen can show the server's own words ("'Her look' is written in the she voice; this persona is he.") without translating anything.

> **THE .NET LENS**
> `PieceError` is a domain exception caught in one place and mapped to `ProblemDetails` — an exception filter, or `app.UseExceptionHandler` with a switch on the exception type. FastAPI has no filter pipeline; the `try` around `assemble` in the handler is the whole mechanism.

10.5 Extracting the pieces that already exist

Typing six personas back in as pieces would be pointless and would drift from what works. The owner's personas are the most trustworthy source of pieces there will ever be, so the feature reads them: `scripts/extract_persona_pieces.py`, run like `seed_dummy.py`.

The rules for a paragraph are simple, deliberately, and there is no model call:

```
RULES: tuple[tuple[PieceSlot, re.Pattern[str]], ...] = (
    (PieceSlot.MARKS, re.compile(r"^(his|her|their)\s+(tattoos?|scars?|piercings?)\b", …)),
    (PieceSlot.HUMOUR, re.compile(r"^(his|her|their)\s+humou?r\b", …)),
    (PieceSlot.PASSIONS, re.compile(r"^(his|her|their)\s+(passions|interests|hobbies)\b", …)),
    (PieceSlot.HOME, re.compile(r"\b(lives? in|penthouse|apartment|…|rides an?)\b", …)),
    (PieceSlot.FAMILY, re.compile(r"\b(son of|daughter of|sister|brother|twin|…)\b", …)),
    …
)
```

The first paragraph is always the look, the first rule a paragraph matches decides its slot, and the work slot takes what is left. A model would have been more flexible and less honest: it would have answered confidently when the rule did not hold, and it would have cost a call per paragraph and given a different answer on a re-run. A rule that is wrong is wrong the same way every time, and the next step catches it.

That step is the **proof**. A persona is saved as pieces only when its pieces, given to the joiner, return its text *exactly*:

```
def rejoins(self) -> bool:
    try:
        return self._rejoined() == self.original.strip()
    except PieceError:
        return False
```

If the suggestion is wrong in a way that changes the order — a work paragraph taken for a personality one — the joiner puts it elsewhere and the comparison fails, and the persona is left out and named. The first run on the owner's six personas is the example: **three of six did not rejoin.** The tattoo rule was `\btattoo`, and the paragraph "His twin sister, Macarena, is a gifted visual artist and tattoo artist." matched it and became a marks paragraph, which sits in the joiner's order *before* the family one. The fix was to anchor the rule to paragraphs that *start* with "His tattoos", and the second run rejoined all six, the roommate variant included. Without the proof, that bug would have shipped as a persona whose sister paragraph came out in the wrong place.

Around the rules: a paragraph several personas share (they share most of them) becomes **one** piece, because pieces are keyed by `(slot, voice, text)` and an existing one is reused, which also makes a second run add nothing; the persona's own name is replaced by the placeholders; every piece is tagged `he`; and the persona itself is only *read*. The owner confirms each persona in a review, and that review is written so it can be tested without a terminal:

```
def review(plan: Plan, ask: Callable[[str], str], say: Callable[[str], None]) -> bool:
```

The script passes `input` and `print`; a test passes a list of answers and a list to collect what was shown. `--dry-run` shows the plans and writes nothing, and `--yes` accepts every suggestion that rejoins.

> **THE .NET LENS**
>
> `ask` and `say` are `Func<string, string>` and `Action<string>`: the seam where .NET code would take an `IConsole` or a `TextReader` / `TextWriter`. Injecting two callables instead of an interface is the Python habit for a seam this small, and a lambda in a test is the whole fake.

10.6 The two clients

Neither client knows how a persona is joined; they choose, and they ask the server to show them.

The terminal (`tui/src/lustjinn_tui/builder.py`) has the slots down the left and the pool of the selected slot on the right, the preview underneath. `Space` picks, `N` writes a new piece in the editor, `V` changes the voice, `R` picks at random, `Ctrl+S` names the persona and makes it. It opens from the library (`B`) and from the new-story form (`Ctrl+B`), and tells the screen that opened it which persona was made, so the library selects it and the form picks it. Its tests run against the same fake server as the rest of the client, which gained the pool and the two join endpoints with the API's own rules in the small.

The web app (`routes/library/personas/from-pieces`) is one card per slot in a column, with the pool opening under the slot, a sheet for a new piece, and the preview as the server's own join, asked again a moment after the pick stops changing. The design was shown to the owner in dark and light before it was built, as every screen is. A small thing worth knowing: the route is `from-pieces` and not `build`, because the repository's `.gitignore` ignores any folder called `build` — a page named that was never committed, which `git status` showed by not listing it.

Both ask for the same minimum, and it is the one rule that lives in the screens and not in the API. The API needs one piece. The screens need a **name** and a **look**: every persona opens with them, and a piece that writes the name needs a name piece. A slot with nothing picked says "needed" or "None picked — optional", and making a persona without the needed ones says what is missing instead of failing. Keeping the rule out of the server is deliberate: anything else that uses the API is not bound by what these two screens think a persona should have.

10.7 Reading and editing the pool

The first version of the pieces could be picked, written in the builder and deleted through the API, but not read whole or changed: the builder showed a piece as a few lines of its text, and the API's save

had no client. The owner had thirty-six pieces and nowhere to see them. The second pass gave them a shelf and an editor in each client, and a description like the other shelves'.

The shelf. The library gains a fourth tab, *Pieces*, with its count. The pool is listed in the persona's own slot order, then by name; a row is the name, the voice, and the description when there is one, falling back to the start of the text. A piece has no history (a persona built from it keeps its own text, so an old version of a piece is not lost the way an old persona is), and the slot cannot change: moving a piece to another slot would change what it means.

The editor. It shows the whole text with `{name}` and `{short}` as written (and two buttons that insert them), the voice, the description with a counter, and for a name piece the full name and the short form. A save carries the version it started from, and a save over a change made elsewhere shows the piece as the server has it now and lets the owner replace it or take it — the same conflict as the other shelves, because the API's 409 carries `current` for pieces too. Deleting asks first and says that personas already built keep their copy.

The terminal client's trick. The library screen is written for entries: a name, a text, a version. Rather than teach it a second kind of thing, the API client maps a piece *onto* an entry (`PieceEntry`, an `EntryFull` with the slot and voice beside it, named "Home · Penthouse (he)") and routes `entries`, `save_entry` and `delete_entry` to `/persona-pieces` when the shelf is `pieces`. The screen then needs only the pieces' own keys — `V` for the voice, `S` for a name piece's short form, `N` asking for a slot and a name. It is the adapter pattern: the screen keeps one model and the client does the translation. There is also no endpoint for one piece, only the pool, so the list carries each piece's whole text and the shelf makes no call per row.

The web client's trick is smaller. Leaving the builder for the editor would lose the pick, which lives in the page's memory. The builder writes its pick to `sessionStorage` on every change (for this tab only), reads it back when it mounts — putting back any piece that was deleted or is in another voice meanwhile — and forgets it once the persona is made. An editor opened from the builder carries `?back=builder`, and Back returns there.

10.8 Testing it

- **The joiner is tested as a function:** order, an empty slot, the home paragraph, more than one piece in a slot, a piece with more than one paragraph, the placeholders, the refusals.
- **The API through `client`:** the pool, the version conflict, a name taken in one voice and not in another, a preview that saves nothing, a build that makes an ordinary persona with its history row, a number added to a taken name, and — the test that states the whole design — that editing or deleting a piece afterwards leaves the persona as it was.
- **Extraction on an invented persona**, so no one's real text is in the repository, including the end-to-end acceptance: pick a persona's pieces in its own order, call the real build endpoint, and get its text back exactly.
- **The clients:** the terminal on its fake server, headless through Textual's pilot; the web app's rules, which need no screen (what is needed, toggling, keeping a pick across a change of voice), as plain Vitest tests in `lib/personaBuilder.spec.ts`.

> **TRY IT**

> Look first, write nothing: `uv run python scripts/extract_persona_pieces.py --dry-run` against your own database shows each persona's paragraphs with the slot suggested for each, and whether the pieces would give the persona back exactly. Then run it without the flag and answer the review. Open the web app's library, the Personas shelf, **Build from pieces**; pick a name and a look, and watch the preview change as you add a home and a pet. In the terminal, `uv run lustjinn-tui`, the library, `b`. Change the voice and see the pieces written for the other one appear.

10.9 What this leaves behind

- A pool of pieces in fixed slots and a joiner that builds a persona in the exact format of the ones that already work, with the persona still plain text.
- A way to turn existing personas into pieces that proves, for each, that nothing was lost.
- A picker in each client, with one rule — a name and a look — that lives in the screens.
- A shelf and an editor for the pieces themselves, with a description, in both clients.
- Two decisions to note. The slot list is closed on purpose, and the voice is a tag on a piece, not a substitution in the text.